CMake by Example

A step-by-step course — from a three-line hello world to a multi-library C project with unit tests, mocking, build options, and a generated config header.

`github.com/TalGadasiSolaredge/Cmake_tut`

CMake by Example — the book

A hands-on, episode-by-episode course that grows one tiny C program from a three-line "hello world" into a multi-library project with unit tests, mocking, build options, and a generated configuration header — learning CMake the whole way up.

Every episode is a **git branch** holding the complete, compiling project as of that step. Read the chapter, then check out the branch and build it:

```
git checkout ep04-menus-library
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
```

Table of contents

Appendices

How to read this book

Each chapter is self-contained and follows the same shape: what you'll build, the concept, the step-by-step changes with real code, the CMake explained, how to build and run it, gotchas, and a recap. Start at Chapter 0.

Chapter 0 — Introduction & Setup

What this book is

CMake has a reputation for being confusing, mostly because tutorials dump a big `CMakeLists.txt` on you and explain nothing. This book does the opposite: we start with the **smallest possible** CMake project — three lines — and add exactly one idea per episode, building a real (if small) C application as we go.

By the end you'll have used, and understood, the CMake features you actually reach for day to day:

- building executables and libraries
- splitting a project into folders and modules
- include paths and the PUBLIC/PRIVATE model
- generators (Ninja vs Visual Studio) and choosing a compiler
- unit testing with cmocka and CTest
- mocking hardware for host-side tests
- build options and preprocessor defines
- generated configuration headers

What CMake actually is (the one idea to hold onto)

CMake does not compile your code. It is a *generator*: it reads your `CMakeLists.txt` and writes out build files for some other tool, which then does the real compiling.

```
CMakeLists.txt ──▶ [ cmake generates ] ──▶ build files ──▶ [ build tool ] ──▶ your program
```

In this book the build tool is **Ninja**. So there are always two steps:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc   # 1. configure (generate)
cmake --build build                                    # 2. build (compile)
```

Chapter 5 digs into what a generator is and how you'd swap Ninja for Visual Studio's MSBuild.

Prerequisites / toolchain

This project is built and tested with:

Tool	What / where (this machine)
CMake	3.15+ (any recent version)
Compiler	gcc — mingw / w64devkit, at `C:\MinGW\w64devkit\bin`
Generator/build tool	Ninja
Test framework	cmocka — a mingw build at `C:\MinGW\cmocka` (from Chapter 6 on)

Put the compiler toolchain on your PATH before running CMake — gcc needs its siblings `as` and `ld`, and CMake needs `ninja`:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
```

If you skip this you'll see `gcc: fatal error: cannot execute 'as'` — that's the symptom of the toolchain not being on PATH (see Chapter 5). Add the line to your `~/.bashrc` so every shell has it.

> The cmocka paths (`C:\MinGW\cmocka`) are hard-coded in the test CMake files because that's where it lives on this machine. On another machine, adjust `CMOCKA_DIR` in `tests/CMakeLists.txt`.

How the episodes work

The `main` branch has this book and the final code. Each **episode is its own branch** with the full project as it stood at the end of that episode:

```
git checkout ep01-hello        # the 3-line starting point
git checkout ep10-lib-plus-lib # the finished project
git checkout main              # back to the latest + this book
```

Read the chapter, check out the branch, build it, poke at it. Onward to Chapter 1.

Chapter 1 — Minimal CMake Hello World

> Build and run the smallest possible CMake project — an executable that prints one line — so you can see the two-step CMake workflow with nothing else in the way. **Branch:** `ep01-hello`

What you'll build

A C program that prints `Hello, World!`, driven by a `CMakeLists.txt` that is only three real lines long. That's the whole project — two files:

File	Purpose
`main.c`	The C source. Prints one line and exits.
`CMakeLists.txt`	Tells CMake what to build.

By the end you'll have configured the project, built it, and run the resulting `build/hello.exe`. More importantly, you'll understand *why* there are two separate steps, because that idea underpins everything else in this book.

Starting from scratch

Everything you need lives in the two files above. There is no `src/` folder, no library, no options — just a source file and the minimal recipe to compile it. We're keeping the surface area tiny on purpose: when the project is this small, there's nowhere for confusion to hide.

Here is the complete `main.c`:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

Nothing CMake-specific here — it's ordinary C. The interesting part is how we ask CMake to turn it into a runnable program.

The concept

The single most important idea in this whole book:

CMake does not compile your code. It is a *generator*.

CMake reads your `CMakeLists.txt` and writes out build files for some *other* tool. That other tool is the one that actually invokes the compiler. In this book the other tool is **Ninja**.

```
CMakeLists.txt ──▶ [ cmake generates ] ──▶ build files ──▶ [ Ninja builds ] ──▶ hello.exe
```

Because of this, there are always **two steps**:

1. **Configure** (generate): CMake reads `CMakeLists.txt` and writes build files into a `build/` folder.
2. **Build** (compile): the build tool reads those generated files and produces the executable.

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc   # 1. configure (generate)
cmake --build build                                   # 2. build (compile)
```

Let's decode that first command, because you'll type it constantly:

Flag	Meaning
`-S .`	**S**ource directory — where `CMakeLists.txt` lives. `.` is the current folder (the project root).
`-B build`	**B**uild directory — where CMake writes everything it generates. It creates `build/` if it doesn't exist.
`-G Ninja`	**G**enerator — which build tool to generate files for. Here, Ninja.
`-D CMAKE_C_COMPILER=gcc`	**D**efine a CMake variable. This one tells CMake to use `gcc` as the C compiler.

The second command, `cmake --build build`, is a portable way to say "run the build tool on the `build/` folder." It just calls Ninja for you, so you don't have to remember Ninja's own command line.

Step by step

1. Get onto the episode branch and put the toolchain on your PATH (details in the Get this episode block below).
2. **Configure once:** `cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc`. This creates and fills the `build/` folder.
3. **Build:** `cmake --build build`. This compiles `main.c` and links it into `build/hello.exe`.
4. **Run:** `./build/hello.exe`.

You only need to re-run step 2 when you change something structural in `CMakeLists.txt`. For everyday code edits you just re-run step 3 — Ninja figures out what changed and recompiles the minimum.

The CMake, explained

Here is the entire `CMakeLists.txt`, exactly as committed on this branch:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Build an executable called "hello" from main.c.
add_executable(hello main.c)
```

Three real lines (the rest are comments). Taken one at a time:

- `cmake_minimum_required(VERSION 3.15)` — declares the oldest CMake version that understands this file. If someone runs an older CMake, they get a clear error instead of a mysterious failure. It should be the first line of every `CMakeLists.txt`.
- `project(hello LANGUAGES C)` — names the project `hello` and tells CMake which languages it uses. `LANGUAGES C` means "this is a C project" — so CMake goes looking for a C compiler (and doesn't waste time hunting for a C++ one). This is the point where CMake detects and validates your compiler.
- `add_executable(hello main.c)` — the payoff line. It says: build an executable **target** named `hello` from the source file `main.c`. The first argument is the target name; everything after is source files. On Windows the produced file gets an `.exe` suffix automatically, so you end up with `hello.exe`.

That's genuinely all CMake needs to build a program.

Build and run

Run the two steps, then the executable:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Expected output:

```
Hello, World!
```

Note the path: with the Ninja generator the executable lands directly at `build/hello.exe`. There is **no** `Debug/` or `Release/` subfolder — that layout is a Visual Studio thing, and we cover it in Chapter 5.

What's autogenerated

You wrote two files. CMake wrote everything else. The entire `build/` folder is generated output — you never edit it by hand and you never commit it. (On this branch, `.gitignore` contains `/build/`, so git ignores it for you.)

The interesting things CMake drops into `build/`:

Path	What it is
`build/CMakeCache.txt`	Cached configuration — the compiler it found, your `-D` options, and more. This is why you configure *once* and build many times: the answers are remembered here.
`build/CMakeFiles/`	CMake's internal bookkeeping — feature-detection results, dependency info, scratch files. You can ignore it.
`build/build.ninja`	The actual build file CMake generated for Ninja. This is the "output" of the generator step — the recipe Ninja follows.
`build/cmake_install.cmake`	The script that would run on `cmake --install` (not something we use yet).

Path	What it is
`build/hello.exe`	Your compiled program, produced by the build step.

Because it's all regenerated, `build/` is completely disposable. Delete it and reconfigure and you get an identical result:

```
rm -rf build
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
```

If your build ever gets into a weird state, wiping `build/` and reconfiguring is the first thing to try.

Gotchas

- **Run `cmake` from the project root**, not from inside a subfolder. `-S .` points at the current directory, so CMake expects to find `CMakeLists.txt` right there. Configuring from the wrong directory means CMake can't find the source (or finds the wrong one).
- **The toolchain must be on your PATH.** `gcc` doesn't work alone — it shells out to its siblings `as` (the assembler) and `ld` (the linker). If they aren't on PATH you'll see an error like `gcc: fatal error: cannot execute 'as'`. Put the toolchain's `bin/` directory on PATH first:

  ```
  export PATH="/c/MinGW/w64devkit/bin:$PATH"
  ```

  Chapter 5 goes into detail on toolchains, generators, and why this matters.

Recap

- CMake is a **generator**, not a compiler. It reads `CMakeLists.txt` and writes build files for another tool (here, Ninja).
- The workflow is always two steps: **configure** (`cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc`) then **build** (`cmake --build build`).
- `-S .` is the source dir, `-B build` is the output dir, `-G` picks the generator, `-D` sets a variable.
- Three lines do the work: `cmake_minimum_required`, `project`, and `add_executable(hello main.c)`.
- The whole `build/` folder is autogenerated and never committed; delete it any time and it regenerates.

Get this episode

```
git checkout ep01-hello
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 2 — A Header and Its Source File

> Split a program into a header (the promise) and a source file (the code), and tell CMake about the new `.c`. **Branch:** `ep02-header-source`

What you'll build

The same `hello` program as Chapter 1, but now it does a little arithmetic through a separate module. We add a tiny `math_utils` "library" — really just one function, `add` — living in its own pair of files:

- `math_utils.h` — the **declaration**: a one-line promise that a function called `add` exists.
- `math_utils.c` — the **definition**: the actual body of `add`.

`main.c` will `#include "math_utils.h"` and print `add(2, 3)`. On the CMake side you'll learn the single most common day-to-day edit: adding a `.c` file to `add_executable`.

Where we left off

Chapter 1 built the smallest possible CMake project. The whole thing was two files. The `CMakeLists.txt` was:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Build an executable called "hello" from main.c.
add_executable(hello main.c)
```

and `main.c` just printed a greeting:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

One source file, one executable, no moving parts. Now we split logic out of `main.c` into its own module — which is where headers enter the story.

The concept

Real programs are not one giant file. You break them into modules so each piece can be understood, tested, and reused on its own. In C, a module is conventionally a **pair** of files:

- A **header** (`.h`) — the *declaration*. It says *what* exists: the name of a function, what arguments it takes, what it returns. It is a promise, not an implementation.
- A **source** (`.c`) — the *definition*. It contains the actual code that fulfills the promise.

Declaration vs. definition. This distinction is the heart of the chapter.

- A *declaration* tells the compiler the shape of something so it knows how to call it: `int add(int a, int b);` . No body, just a signature ending in a semicolon.
- A *definition* is the real thing, with a body:

```
int add(int a, int b)
{
    return a + b;
}
```

Why headers exist. When the compiler processes `main.c` , it compiles that file *on its own*. It never sees the inside of `math_utils.c` . So how does it know that `add` takes two `int` s and returns an `int` ? Because `main.c` includes `math_utils.h` , and the header carries the declaration. The header lets one file learn how to *call* a function without ever seeing its *body*. The body gets connected later, by the linker.

That is the mental model to hold onto:

```
math_utils.h   →  the promise   (declaration)   →  included by anyone who calls add
math_utils.c   →  the code      (definition)     →  compiled once, linked in
main.c         →  the caller    (includes the promise, calls add)
```

Include guards. A header often gets included more than once in a single compilation — directly and again indirectly through some other header. If the compiler pasted the same declarations in twice, you'd get "redefinition" errors. The classic fix is the **include guard**:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/* ... the header's contents ... */

#endif /* MATH_UTILS_H */
```

Read it as: "if the symbol `MATH_UTILS_H` is *not* already defined, define it and process everything down to `#endif` ." The first time the file is included, the symbol doesn't exist, so the body is used and the symbol gets defined. Every later time, the symbol is already defined, so the preprocessor skips straight to `#endif` . The result: the header's contents appear exactly once, no matter how many times it's included.

Step by step

1. Create the header — the promise. This is `math_utils.h`:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/* Returns the sum of a and b. */
int add(int a, int b);

#endif /* MATH_UTILS_H */
```

Note there's no body — just the declaration `int add(int a, int b);` wrapped in an include guard. Anyone who includes this file now knows how to call `add`.

2. Create the source — the code. This is `math_utils.c`:

```
#include "math_utils.h"

int add(int a, int b)
{
    return a + b;
}
```

The source file includes its *own* header. That's deliberate: it lets the compiler check that the definition here matches the declaration in the header (same name, same parameters, same return type). If they ever drift apart, you get a compile error instead of a subtle bug.

3. Call it from `main.c`. We include the header and use `add`:

```
#include <stdio.h>
#include "math_utils.h"

int main(void)
{
    printf("Hello, World!\n");
    printf("2 + 3 = %d\n", add(2, 3));
    return 0;
}
```

Two things changed from Chapter 1: the new `#include "math_utils.h"` and the new `printf` line that calls `add(2, 3)`. Note the quotes in `#include "math_utils.h"` — quotes are for *your* headers (looked up starting in the file's own directory), while angle brackets like `#include <stdio.h>` are for system/library headers.

4. Tell CMake about the new `.c`. More on this next, but the edit is a single line in `CMakeLists.txt`: add `math_utils.c` to `add_executable`.

The CMake, explained

Here is the complete `CMakeLists.txt` for this episode:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Build an executable called "hello" from these source files.
# Add every .c file here. Headers (.h) do NOT go in this list.
add_executable(hello main.c math_utils.c)
```

The only change from Chapter 1 is the last line: `add_executable(hello main.c)` became `add_executable(hello main.c math_utils.c)`.

This is the key CMake lesson of the chapter: in `add_executable`, you list **only the `.c` files** — every source file that needs to be compiled. You do **not** list `.h` files. Ever.

Why? Because headers are not compiled on their own. They get pulled into a compilation automatically wherever a `.c` file says `#include`. When `math_utils.c` and `main.c` each `#include "math_utils.h"`, the preprocessor splices the header's text into those files before compilation. CMake doesn't need to be told about the header — the `#include` directives handle it.

Listing a `.h` file in `add_executable` is simply pointless: it doesn't cause the header to be compiled (headers aren't compilation units), and it doesn't create any dependency you don't already get from the `#include`. It's a no-op that only adds noise. So the rule is blunt and easy to remember:

> `add_executable` gets `.c` files, one per module. Headers travel via `#include`, never in this list.

What about finding the header? At this stage every file lives in the project root, side by side. When the compiler processes `main.c` and hits `#include "math_utils.h"`, it looks in the file's own directory first — and finds it, because they're in the same folder. So we need **no include-path setting yet**. That changes in Chapter 3, when we move headers into an `inc/` folder and have to tell the compiler where to look.

Build and run

Same two steps as always — configure, then build:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Expected output:

```
Hello, World!
2 + 3 = 5
```

If you had already configured this project in Chapter 1, you don't have to re-run the `cmake -S . -B build ...` line by hand. CMake tracks its own input files, so the next `cmake --build build` notices that `CMakeLists.txt` changed, re-generates the build files automatically, then compiles. Either way you land on the same result.

What's autogenerated

Nothing new compared to Chapter 1. As before, everything CMake produces lands in the `build/` directory — the generated Ninja files, the object files, and the final `hello.exe`. It's all disposable: delete `build/` and a fresh `cmake -S . -B build ...` regenerates it from scratch. That's exactly why `build/` is in `.gitignore` and never committed. Adding a second source file didn't introduce any new *kind* of generated output — just one more object file inside `build/`.

Gotchas

- **Forgetting to add the new `.c` to `add_executable`.** This is the classic first mistake. If you add `math_utils.c` to the project but leave `add_executable(hello main.c)` unchanged, the code *compiles* — `main.c` sees the declaration from the header, so the compiler is happy — but it fails at the **link** step with something like `undefined reference to 'add'`. The linker is telling you: someone promised `add` exists, but I was never given the file that defines it. The fix is to list `math_utils.c` in `add_executable`.
- **Putting a `.h` file in `add_executable`.** Harmless but pointless. Writing `add_executable(hello main.c math_utils.c math_utils.h)` doesn't compile the header (headers aren't compilation units) and adds no dependency you didn't already have from `#include`. Leave headers out.
- **Missing or mismatched include guards.** If you forget the `#ifndef / #define / #endif` guard and the header ends up included twice in one file, you'll hit "redefinition" errors. Every header should have a guard whose macro name is unique to that file (here, `MATH_UTILS_H`).
- **Angle brackets vs. quotes.** Use `#include "math_utils.h"` (quotes) for your own project headers and `#include <stdio.h>` (angle brackets) for system headers. Getting these backwards can make the compiler fail to find your header.

Recap

- A C module is a **pair**: a header (`.h`) holding the **declaration** — the promise that a function exists — and a source (`.c`) holding the **definition** — the actual code.
- Headers exist so one file can learn how to *call* a function without seeing its *body*; the body is connected later by the linker.
- **Include guards** (`#ifndef / #define / #endif`) stop a header's contents from being pasted in more than once.
- In `add_executable`, list **only `.c` files** — one per module. **Never** list `.h` files; headers are pulled in automatically through `#include`, and listing them does nothing.
- Forgetting to add a new `.c` to `add_executable` compiles fine but fails to link with `undefined reference`.
- With all files in the project root, the compiler finds `math_utils.h` automatically — no include-path setting needed yet.

Get this episode

```
git checkout ep02-header-source
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 3 — Organizing into Inc/ and Src/

> Move headers into `Inc/` and sources into `Src/`, then learn the one new command that keeps `#include` working after the move: `target_include_directories`. **Branch:** `ep03-inc-src-layout`

What you'll build

The exact same program as Chapter 2 — it still prints a greeting and adds two numbers — but the files are now organized the way real C projects lay them out: public headers in an `Inc/` folder, source files in a `Src/` folder.

```
cmake-course/
├── CMakeLists.txt
├── .gitignore
├── Inc/
│   └── math_utils.h
└── Src/
    ├── main.c
    └── math_utils.c
```

Nothing about *what the program does* changes. This episode is entirely about **layout**, and about the one CMake command you need to add so the compiler can still find your header after you move it.

Where we left off

In Chapter 2 every file sat flat in the repository root — `main.c`, `math_utils.c`, and `math_utils.h` were all next to each other:

```
cmake-course/
├── CMakeLists.txt
├── main.c
├── math_utils.c
└── math_utils.h
```

And the CMake was short:

```
add_executable(hello main.c math_utils.c)
```

That worked, and importantly `#include "math_utils.h"` in `main.c` resolved **for free**. Why? Because the header was sitting right next to the `.c` files that included it. A quoted `#include` looks in the folder of the current file first, so the compiler found `math_utils.h` without us telling it anything.

The moment we move the header into its own folder, that free lookup stops working — and that is the whole lesson of this chapter.

The concept

The compiler's header search path

When the compiler hits a line like:

```
#include "math_utils.h"
```

it has to go find a file called `math_utils.h` on disk. It does not search your whole computer. It searches a specific, ordered list of folders called the **header search path** (also called the *include path*).

For a quoted include (`"..."`), the compiler looks **next to the current source file first**, then falls through to the directories on the search path. In Chapter 2 the "next to the current file" step was enough, because the header lived in the same folder as `main.c` .

Now `main.c` is in `Src/` and `math_utils.h` is in `Inc/` . They are in different folders. "Next to `main.c` " no longer contains the header, and `Inc/` is not on the search path yet — so the include fails.

Putting `Inc/` back on the path

The fix is to add `Inc/` to the header search path. In CMake, you do that with one command:

```
target_include_directories(hello PRIVATE Inc)
```

Read it as: *"When building the target `hello` , add the folder `Inc` to the list of places the compiler looks for headers."* Now `#include "math_utils.h"` resolves again, because `Inc/` is on the path and `math_utils.h` lives there.

Headers are still not build sources

This is worth repeating from Chapter 2, because moving the header into `Inc/` can make it *feel* like it should now be "added" somewhere new. It should not.

Look at `add_executable` — it lists only `.c` files. `math_utils.h` appears **nowhere** as a thing to compile. What we did with `target_include_directories` is completely different: we told the compiler **where to find** the header, not to compile it. Headers are found and pasted in by `#include` ; they are never compiled on their own.

So the two jobs are cleanly separated:

- `add_executable(hello Src/main.c Src/math_utils.c)` — *what to compile* (the `.c` files).
- `target_include_directories(hello PRIVATE Inc)` — *where to look* for the `.h` files those sources include.

What `PRIVATE` means here

`target_include_directories` takes a keyword — here `PRIVATE` . It answers the question: *"Who gets to use this include directory?"*

`PRIVATE` means: **this include directory is used only when building `hello` itself.** Nobody else. That is exactly right for an executable — nothing links *against* `hello` , so there is no "anyone else" to share the path with.

The keyword only starts to earn its keep once you have libraries, where one target is built *and then used by* another target. When we build our first library in Chapter 4, we'll see why a library might want `PUBLIC` (so code that links the library also gets its headers on the search path) versus `PRIVATE` (headers the library needs internally but that its users should not see). For an executable, `PRIVATE` is the natural choice, so that's what we use.

Step by step

1. Create the two folders and move the files.

- `math_utils.h` → `Inc/math_utils.h`
- `main.c` → `Src/main.c`
- `math_utils.c` → `Src/math_utils.c`

The *contents* of the files do not change at all — only their location. Here they are for reference.

`Inc/math_utils.h`:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/* Returns the sum of a and b. */
int add(int a, int b);

#endif /* MATH_UTILS_H */
```

`Src/main.c`:

```
#include <stdio.h>
#include "math_utils.h"

int main(void)
{
    printf("Hello, World!\n");
    printf("2 + 3 = %d\n", add(2, 3));
    return 0;
}
```

`Src/math_utils.c`:

```
#include "math_utils.h"

int add(int a, int b)
{
    return a + b;
}
```

Notice that `main.c` still writes `#include "math_utils.h"` — just the bare file name, not `#include "../Inc/math_utils.h"`. We do **not** hard-code the folder into the source. Instead we let CMake add `Inc/` to the search path, which keeps the source clean and portable. That's the whole point of an include directory.

2. Update `CMakeLists.txt` so the source paths point into `Src/`, and add the new `target_include_directories` line so `Inc/` is on the header search path.

The CMake, explained

Here is the complete `CMakeLists.txt` for this episode:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Build an executable called "hello" from these source files (all under Src/).
add_executable(hello
    Src/main.c
    Src/math_utils.c
)

# Headers live under Inc/ -- tell the compiler where to find them so that
# #include "math_utils.h" resolves even though the .h is in a different folder.
target_include_directories(hello PRIVATE Inc)
```

Two things changed from Chapter 2:

- **The source paths are now folder-qualified.** `add_executable` lists `Src/main.c` and `Src/math_utils.c` instead of the bare `main.c` / `math_utils.c`. These paths are relative to the folder containing `CMakeLists.txt` (the project root), so CMake resolves them straight to the files inside `Src/`. We also spread the list across multiple lines — CMake does not care about the whitespace, and it reads more clearly once you have a couple of sources.
- **A new command appeared:** `target_include_directories(hello PRIVATE Inc)`. This is the line doing the real work of this chapter. `Inc` is again relative to the project root, so it points at our `Inc/` folder. Adding it to `hello`'s header search path is what makes `#include "math_utils.h"` resolve again.

Note the shape of the command: it is `target_`-something and its first argument is a target (`hello`). CMake has a whole family of `target_*` commands (`target_include_directories`, `target_link_libraries`, `target_compile_definitions`, ...) that all attach a setting **to a specific target** rather than to the whole project. This is the modern, recommended way to configure things in CMake, and you'll meet more of these commands in later chapters.

Build and run

The build steps are identical to before — configure, then build. From the project root:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Expected output:

```
Hello, World!
2 + 3 = 5
```

Exactly what Chapter 2 printed — because only the layout changed, not the behavior. If you get that output, the reorganization worked and CMake found the header in its new home.

What's autogenerated

Nothing new this episode. As in Chapter 1, `cmake -S . -B build` fills the `build/` folder with generated files — `build.ninja`, `CMakeCache.txt`, the `CMakeFiles/` bookkeeping directory — and `cmake --build build` produces `hello.exe` there. You still never edit anything under `build/`, and `build/` is still git-ignored. Moving your own files into `Inc/` and `Src/` doesn't add any new categories of generated output; it just changes where *your* files live.

Gotchas

- **Forgetting `target_include_directories` after moving the header.** This is the classic first stumble with this layout. You move `math_utils.h` into `Inc/`, update the `Src/` paths in `add_executable`, build — and it fails with:

  ```
  fatal error: math_utils.h: No such file or directory
  ```

  The compiler is telling you plainly: it looked, and `Inc/` was not on the search path, so it couldn't find the header. The fix is the one line this chapter is about — `target_include_directories(hello PRIVATE Inc)`. If you see a `No such file or directory` on a header, "is its folder on the include path?" is the first question to ask.

- **Trying to "fix" it in the source instead.** It's tempting to change the include to `#include "../Inc/math_utils.h"` to make the error go away. Don't. That hard-codes the relative layout into your source and breaks the moment the file moves again. The include directory belongs in the build configuration (`CMakeLists.txt`), not baked into every `#include`.

- **Adding the `.h` to `add_executable`.** Also tempting, also wrong — and it won't even fix the error. Headers are never listed as sources; the search path is what resolves them.

Recap

- Real projects separate **headers** (`Inc/`) from **sources** (`Src/`); this episode adopts that layout without changing the program's behavior.
- A quoted `#include` is resolved against the compiler's **header search path**. It searches next to the current file first, which is why a flat layout "just worked" in Chapter 2.

- Once the header lives in a different folder, you must add that folder to the search path with `target_include_directories(hello PRIVATE Inc)`.
- Headers are still **never** build sources. `add_executable` lists only `.c` files; `target_include_directories` tells the compiler where to *find* the `.h` files, not to compile them.
- `PRIVATE` means the include directory is used only to build `hello` — the distinction between `PUBLIC` and `PRIVATE` becomes important once we have libraries in Chapter 4.

Get this episode

```
git checkout ep03-inc-src-layout
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 4 — Your First Library Module

> Split a self-contained feature into its own folder that builds as a **library**, then link it into the app — the pattern every large CMake project is built from. **Branch:** `ep04-menus-library`

What you'll build

So far our project has been one executable, `hello`, compiled from a flat list of source files in the top-level `CMakeLists.txt`. That works when you have three files. It stops working when you have three hundred.

In this episode we carve out a new feature — a couple of on-screen menus — and give it its own home: a `menus/` folder with its **own** `CMakeLists.txt`. That folder describes how to build itself, producing a small **static library**. The top-level file just says "go build the `menus` folder" and "link it into the app." Crucially, the top-level file never lists a single menu source file and never mentions the menu headers' folder — and yet `main.c` can include and call the menu functions. Understanding *why* that works is the whole point of the chapter.

When you run it, the program prints the greeting and math from before, then two menus:

```
Hello, World!
2 + 3 = 5

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

Where we left off

At the end of Chapter 3 the project used an `Inc/` + `Src/` layout: headers in `Inc/`, sources in `Src/`, and one `target_include_directories(hello PRIVATE Inc)` so `#include "math_utils.h"` resolved. The tree looked like this:

```
cmake-course/
├── CMakeLists.txt
├── Inc/
│   └── math_utils.h
└── Src/
    ├── main.c
    └── math_utils.c
```

Everything — every source file, every include path — was declared in the single top-level `CMakeLists.txt`. That is exactly the thing that doesn't scale, and what we fix now.

The concept

Three ideas come together in this chapter.

1. A library is just "compiled code you don't run directly." Up to now we built an *executable* with `add_executable` — something with a `main()` you can run. A **library** is compiled object code with no `main()`, meant to be *linked into* an executable (or another library). You build one with `add_library`. By default CMake makes a **static** library: an archive of `.o` object files that gets copied into the final executable at link time. Our `menus` folder becomes such a library.

2. Each folder owns its own build description. A subfolder can have its own `CMakeLists.txt`. The parent pulls it in with `add_subdirectory(menus)`, and CMake runs that child file as part of configuring the project. The top-level file therefore does **not** need to know the menu source files exist — the `menus/CMakeLists.txt` lists them. This is the scaling pattern: big projects stay manageable by **dividing** responsibility across folders, not by growing one giant file. Each module is self-contained and could, in principle, be dropped into another project.

3. Usage requirements: PUBLIC / PRIVATE / INTERFACE. When a target declares an include directory (or a compile definition, or a linked library), CMake lets it say *who else needs it*. This is the single most important idea in modern CMake. We'll declare the `menus` include folder **PUBLIC**, which means "I need this to build myself, **and** anything that links me inherits it automatically." That inheritance is why `main.c` can `#include "main_menu.h"` even though the top-level `CMakeLists.txt` never mentions `menus/Inc`.

Step by step

The new folder is self-contained: its own headers under `Inc/`, its own sources under `Src/`, and its own `CMakeLists.txt`. The full project now looks like this:

```
cmake-course/
├── CMakeLists.txt          # top level: builds the app, pulls in menus/
├── Inc/
│   └── math_utils.h        # the app's own headers  (PRIVATE to hello)
├── Src/
│   ├── main.c
│   └── math_utils.c
└── menus/                  # a self-contained module → becomes a library
    ├── CMakeLists.txt      # describes how to build the menus library
    ├── Inc/
    │   ├── main_menu.h      # menus' public headers (PUBLIC → hello inherits)
    │   └── settings_menu.h
    └── Src/
        ├── main_menu.c
        └── settings_menu.c
```

Notice there are now **two** `Inc/` folders. The top-level `Inc/` holds `math_utils.h`, reachable because `hello` sets `target_include_directories(hello PRIVATE Inc)` on itself. The `menus/Inc/` holds the menu headers — and `main.c` reaches

those only because linking `menus` drags in its PUBLIC include path. Keep the two straight; the PUBLIC lesson lives in the difference.

The menu headers

Each header just declares one function. Standard include guards, nothing more.

`menus/Inc/main_menu.h`:

```
#ifndef MAIN_MENU_H
#define MAIN_MENU_H

/* Prints the main menu to stdout. */
void print_main_menu(void);

#endif /* MAIN_MENU_H */
```

`menus/Inc/settings_menu.h`:

```
#ifndef SETTINGS_MENU_H
#define SETTINGS_MENU_H

/* Prints the settings menu to stdout. */
void print_settings_menu(void);

#endif /* SETTINGS_MENU_H */
```

The menu sources

Each source includes its own header and implements the one function.

`menus/Src/main_menu.c`:

```
#include <stdio.h>
#include "main_menu.h"

void print_main_menu(void)
{
    printf("=== Main Menu ===\n");
    printf("1) Start\n");
    printf("2) Settings\n");
    printf("3) Quit\n");
}
```

`menus/Src/settings_menu.c`:

```
#include <stdio.h>
#include "settings_menu.h"

void print_settings_menu(void)
{
    printf("=== Settings ===\n");
    printf("1) Volume\n");
    printf("2) Brightness\n");
    printf("3) Back\n");
}
```

The module's own CMakeLists.txt

This is the new file that makes `menus/` a library. It lives at `menus/CMakeLists.txt`:

```
# --- The "menus" module ---
# Bundle this folder's sources into a library called "menus".
add_library(menus
    Src/main_menu.c
    Src/settings_menu.c
)

# PUBLIC means: this include dir is used both when building "menus" itself
# AND automatically added to anything that links against "menus".
# So the main app can #include "main_menu.h" without repeating this path.
target_include_directories(menus PUBLIC Inc)
```

Two things worth noticing. The paths (`Src/main_menu.c` , `Inc`) are relative to **this** file's folder, `menus/` — each `CMakeLists.txt` thinks in terms of its own directory. And only the `.c` files are listed; the headers are nowhere in `add_library` (more on that in Gotchas).

Wiring it into the app

Finally the top-level `CMakeLists.txt` grows three things: it pulls in the subfolder, and it links the resulting library into `hello`.

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Pull in the "menus" subfolder. CMake runs menus/CMakeLists.txt,
# which defines the "menus" library target.
add_subdirectory(menus)

# Build an executable called "hello" from these source files (all under Src/).
add_executable(hello
    Src/main.c
    Src/math_utils.c
)

# Tell the compiler where to find this project's own headers (Inc/).
target_include_directories(hello PRIVATE Inc)

# Link the "menus" library into the app. Because menus' Inc was marked PUBLIC,
# the app automatically gets menus' header search path too.
target_link_libraries(hello PRIVATE menus)
```

Calling the menus from main

`Src/main.c` now includes the menu headers and calls the two functions:

```
#include <stdio.h>
#include "math_utils.h"
#include "main_menu.h"
#include "settings_menu.h"

int main(void)
{
    printf("Hello, World!\n");
    printf("2 + 3 = %d\n\n", add(2, 3));

    print_main_menu();
    printf("\n");
    print_settings_menu();
    return 0;
}
```

Look at those two includes: `main_menu.h` and `settings_menu.h`. They live in `menus/Inc/`. The top-level `CMakeLists.txt` never says `menus/Inc` anywhere. The include still resolves — that's PUBLIC doing its job.

The CMake, explained

`add_library` vs `add_executable`

```
add_library(menus
    Src/main_menu.c
    Src/settings_menu.c
)
```

Same shape as `add_executable`: a target name followed by source files. The difference is the *product*. `add_executable(hello ...)` produces a runnable program; `add_library(menus ...)` produces a library — here a static archive, because we didn't ask for anything else. There's no `main()` in the menu sources, and that's fine: libraries aren't run, they're linked.

`add_subdirectory(menus)` — the scaling move

```
add_subdirectory(menus)
```

This tells CMake: "there is another `CMakeLists.txt` in the `menus` folder — go run it." When CMake configures the project, it descends into `menus/`, executes `menus/CMakeLists.txt`, and comes back knowing about the `menus` target it defined there.

This is why the top-level file stays short. It does **not** list `main_menu.c` or `settings_menu.c` — the menus folder owns that knowledge. Add a tenth source file to the module later and only `menus/CMakeLists.txt` changes; the top level never notices. Multiply that across dozens of modules and you see the pattern: you scale a build by **dividing** it into folders that each describe themselves, not by piling everything into one file.

PUBLIC vs PRIVATE vs INTERFACE — usage requirements

```
target_include_directories(menus PUBLIC Inc)
```

The keyword before the path answers one question: *besides me, who else needs this?* CMake calls these **usage requirements**. Here's the whole model:

Keyword	Do I (the target) need it to build?	Do my consumers (who link me) inherit it?	Typical use
`PRIVATE`	Yes	No	An implementation detail I use internally but don't expose in my headers.
`PUBLIC`	Yes	Yes	Something I use **and** that appears in my public headers, so consumers need it too.
`INTERFACE`	No	Yes	I don't compile it myself but consumers must have it — e.g. a header-only library.

Our `menus` library marks its `Inc` folder **PUBLIC** because two different parties need that path:

1. `menus` **itself** — `main_menu.c` does `#include "main_menu.h"`, and that header is in `Inc`. So `menus` needs `Inc` to compile.

2. **Anything that links `menus`** — `hello` links it and its `main.c` includes `main_menu.h` too. Because the path is PUBLIC, CMake **propagates** it: when you write `target_link_libraries(hello PRIVATE menus)`, `hello` silently inherits `menus/Inc` on its own include path.

Compare with `hello`'s own line, `target_include_directories(hello PRIVATE Inc)`. That is PRIVATE because nothing links `hello` — it's the top of the chain, so there are no consumers to inherit anything. `hello` needs `Inc` (for `math_utils.h`) and no one else does.

If we had marked the menus include dir PRIVATE instead, `menus` would still build fine on its own — but `hello` would **not** inherit `menus/Inc`, and `#include "main_menu.h"` in `main.c` would fail with "file not found." The single word PUBLIC is what makes the module usable by its consumers.

Only `.c` files go in `add_library`

The `add_library(menus ...)` call lists `main_menu.c` and `settings_menu.c` and nothing else. Headers are **not** listed there and don't need to be. The compiler finds headers through the include path (the `Inc` folder we set with `target_include_directories`), not through the source list. Listing a `.h` in `add_library` doesn't cause it to be compiled or otherwise "activate" it — it's inert (see Gotchas).

Build and run

Same two-step flow as always — configure, then build:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

The configure step now processes two `CMakeLists.txt` files (top level, then `menus/` via `add_subdirectory`). The build step compiles the menu sources into the `menus` library, compiles the app sources, and links the library into `hello`. Running it prints:

```
Hello, World!
2 + 3 = 5

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

The two blank lines are deliberate: the `\n\n` after `2 + 3 = 5` separates the math from the first menu, and the `printf("\n")` between the calls separates the two menus.

What's autogenerated

Because `menus/` is now a subdirectory with its own target, CMake mirrors that structure inside `build/`. Alongside the `hello.exe` executable, the build now produces a static library:

```
build/
├── hello.exe           # the app
└── menus/
    └── libmenus.a      # the static library built from the menus module
```

`libmenus.a` is the compiled `menus` library — an archive of the object files for `main_menu.c` and `settings_menu.c`. The `lib` prefix and `.a` extension are the standard naming for a static library with gcc/MinGW; CMake derives that name from the target name `menus` automatically — you never spell out `libmenus.a` anywhere. At link time its contents are folded into `hello.exe`, which is why the running program has the menu code even though you launched only the executable.

Everything else in `build/` — the Ninja files, the `CMakeCache.txt`, the `CMakeFiles/` bookkeeping — is generated exactly as in earlier chapters. As always, `build/` is disposable: delete it and re-run the two commands to regenerate it from scratch.

Gotchas

- `add_subdirectory` **must come before you use the target it defines.** CMake reads the file top to bottom. `add_subdirectory(menus)` is what creates the `menus` target, so it has to appear **above** the `target_link_libraries(hello PRIVATE menus)` that references it. Put the link line first and CMake errors out with something like "Cannot specify link libraries for target 'hello' which is not built by this project" / unknown target `menus`.
- **Putting headers in** `add_library` **does nothing useful.** Adding `Inc/main_menu.h` to the `add_library(menus ...)` list won't make the header "available" to anyone — sources find headers through the *include path*, not the source list. Set the folder with `target_include_directories` (as we did) and leave headers out of `add_library`.
- **Forgetting** `target_link_libraries` **→ undefined references.** If you `add_subdirectory(menus)` but never link it into `hello`, the compile step still succeeds — the headers are found, so `main.c` compiles — but the **link** step fails with `undefined reference to 'print_main_menu'` (and `print_settings_menu`). The declarations were visible; the actual compiled code was never pulled in. The fix is the `target_link_libraries(hello PRIVATE menus)` line.
- **Include dir PRIVATE instead of PUBLIC → header not found.** As covered above: mark `menus`'s include dir PRIVATE and `hello` won't inherit it, so `#include "main_menu.h"` in `main.c` fails. PUBLIC is what propagates the path to consumers.
- **Relative paths are relative to each file's own folder.** Inside `menus/CMakeLists.txt`, `Src/main_menu.c` and `Inc` mean `menus/Src/main_menu.c` and `menus/Inc`. Don't write the paths as if you were in the top-level file.

Recap

- `add_library` builds a **library** (a static archive by default) rather than a runnable executable — compiled code meant to be linked into something else.
- `add_subdirectory(menus)` runs `menus/CMakeLists.txt`. The module owns its own build description; the top-level file never lists the menu sources. This is how builds scale — by **dividing** into self-describing folders.

- Usage requirements (**PRIVATE / PUBLIC / INTERFACE**) control who inherits an include path. `menus` marked its `Inc` **PUBLIC**, so linking `menus` gives `hello` that header path for free — which is why `main.c` includes `main_menu.h` with no mention of `menus/Inc` at the top level.
- Only `.c` files go in `add_library`; headers are found via the include directory, not the source list.
- The build now also emits the static library `build/menus/libmenus.a`.

Get this episode

```
git checkout ep04-menus-library
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 5 — Generators and Commands

> This episode has almost no new *code* — instead we nail down the single most important idea in CMake (**it doesn't compile; it generates**), formally adopt **gcc + Ninja** as the repo's toolchain, and add two reference docs so you never have to guess a command again. **Branch:** `ep05-generators-commands`

What you'll build

Nothing new gets compiled this episode. The program from Chapter 4 still builds and runs exactly the same. What changes is your *understanding* and your *paperwork*:

- You'll learn what a **generator** actually is, and why CMake needs one.
- You'll contrast **Ninja** and **Visual Studio** as two generators for the same `CMakeLists.txt`, and see why this book uses Ninja.
- You'll understand the **cache lock**: why the compiler and generator are chosen *once* and how to change your mind.
- You'll meet the classic `cannot execute 'as'` error and fix it for good.
- You'll get two new files committed to the repo: `README.md` (project overview) and `COMMANDS.md` (the full command reference).

Where we left off

At the end of Chapter 4 the project looked like this: a `hello` executable built from `Src/`, plus a `menus` library pulled in with `add_subdirectory` and linked with `target_link_libraries`. The root `CMakeLists.txt` is unchanged this episode:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name and the language(s) it uses.
project(hello LANGUAGES C)

# Pull in the "menus" subfolder. CMake runs menus/CMakeLists.txt,
# which defines the "menus" library target.
add_subdirectory(menus)

# Build an executable called "hello" from these source files (all under Src/).
add_executable(hello
    Src/main.c
    Src/math_utils.c
)

# Tell the compiler where to find this project's own headers (Inc/).
target_include_directories(hello PRIVATE Inc)

# Link the "menus" library into the app. Because menus' Inc was marked PUBLIC,
# the app automatically gets menus' header search path too.
target_link_libraries(hello PRIVATE menus)
```

We've been typing this command since Chapter 1 without really explaining the back half of it:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
```

Time to explain the `-G Ninja` and `-D CMAKE_C_COMPILER=gcc` parts properly.

The concept

CMake does not compile your code

This is *the* idea to hold onto. CMake never turns a `.c` file into an executable. It is a **generator**: it reads your `CMakeLists.txt` and writes out build files for some *other* tool, and that tool does the actual compiling.

```
CMakeLists.txt ──▶ [ cmake generates ] ──▶ build files ──▶ [ build tool compiles ] ──▶ hello.exe
```

That's why every build is two steps, not one:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc   # 1. configure = generate build files
cmake --build build                                    # 2. build     = the tool compiles
```

Step 1 produces build files. Step 2 hands those files to the build tool. The `-G` flag is where you tell CMake **which build tool to generate for**.

Two generators, same CMakeLists.txt

The beauty of writing `CMakeLists.txt` instead of hand-writing build files is that *one* description can target *many* build systems. Pick the target with `-G`:

	`-G "Visual Studio 17 2022"`	`-G Ninja`
CMake writes	`.sln` + `.vcxproj` files	`build.ninja`
Compiler used	MSVC (`cl.exe`) via MSBuild	whatever you point at (gcc here)
Build tool	MSBuild	Ninja
Configs	multi-config (Debug/Release in one tree)	single-config (chosen at configure)
The exe lands at	`build/Debug/hello.exe`	`build/hello.exe`
Feel	IDE-oriented, heavier	fast, minimal, cross-platform

Same source, same `CMakeLists.txt` — only `-G` differs, and CMake emits a completely different set of build files.

Why this book uses Ninja

We standardize on Ninja for a few concrete reasons:

- **Speed**, especially *incremental* builds. Change one file and Ninja rebuilds only what depends on it, near-instantly.
- **Cross-platform.** The same generator works on Windows, Linux, and macOS, so the commands in this book don't change per OS.
- **Clean progress output.** Ninja prints a tidy `[n/m]` counter (`[3/4] Building C object …`) so you can see exactly how far along a build is.
- **It's machine-generated.** `build.ninja` is meant to be *written by a tool*, not by a human — its syntax is deliberately dumb and verbose. That's the whole point of a generator: you write the readable `CMakeLists.txt` , and CMake writes the tedious `build.ninja` for you. You should never open `build.ninja` to edit it by hand.

The cache lock

Both `-G <generator>` and `-D CMAKE_C_COMPILER=<gcc|clang>` are decisions CMake makes **at the first configure** and then **caches** in `build/CMakeCache.txt` . On every later `cmake --build build` , CMake reuses the cached choices — it does *not* re-read those flags. That's a feature: the toolchain stays consistent for the life of the `build/` directory.

The catch: you can't change your mind by just re-running configure with different flags. To switch compiler or generator you must **start fresh** — delete the build directory and reconfigure:

```
rm -rf build
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=clang     # different compiler
```

```
rm -rf build
cmake -S . -B build -G "Visual Studio 17 2022"             # different generator (MSVC)
```

If you skip the `rm -rf build` , CMake keeps the old cached toolchain and quietly ignores your new flags.

The toolchain must be on your PATH

gcc from mingw / w64devkit isn't a single self-contained program. `gcc.exe` drives a small family of tools that live next to it — the assembler `as` and the linker `ld`. CMake also needs `ninja`. All of them must be findable on your `PATH`:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
```

Skip this and you hit the single most common first-time error:

```
gcc: fatal error: cannot execute 'as': CreateProcess: No such file or directory
```

Read that carefully — it's confusing until it clicks. gcc *did* run, so gcc itself was found (CMake invoked it by full path). But gcc then tried to run its sibling `as` **by bare name**, expecting to find it on PATH, and couldn't, because the `w64devkit\bin` folder isn't on PATH. gcc found itself but not its siblings.

The fix is to put that folder on PATH (the `export` line above), and add it to your `~/.bashrc` so every new shell has it. One more step is easy to forget: if you already tried to configure and it failed, CMake has **cached that the compiler is broken**. Fixing PATH isn't enough — you must also nuke the build dir so CMake re-tests the now-working compiler:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"   # fix PATH (and add to ~/.bashrc)
rm -rf build                                 # forget the "compiler broken" cache
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
```

Step by step

The build graph does **not** change this episode. There are no new targets, no edits to any `CMakeLists.txt`. We only:

1. **Add `README.md`** — a top-level overview of the whole project: what it is, how episodes-as-branches work, the toolchain, and a quickstart. It's the first thing a newcomer to the repo reads.
2. **Add `COMMANDS.md`** — a complete, copy-pasteable command reference: configure, build, run, test, and clean. This is the file you'll keep open in a tab for the rest of the book.
3. **Formally adopt gcc + Ninja** as *the* toolchain for every episode. From here on, the book assumes this combination.

That's the entire code delta — two new documentation files:

```
COMMANDS.md | 69 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
README.md   | 39 +++++++++++++++++++++++++++++++++++
2 files changed, 108 insertions(+)
```

The CMake, explained

There's no CMake *code* to explain this episode — but there is a command reference to internalize. The two new docs are the deliverable, so let's read the real content.

`README.md` — the project overview

`README.md` orients anyone landing on the repo. Its key sections:

> This repo is built with **gcc (mingw / w64devkit) + Ninja**. That single toolchain builds every episode, including the cmocka unit tests introduced later. Episode 5 explains what a *generator* is and how you'd use Visual Studio / MSVC instead.

It reminds you to put the compiler on PATH…

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
```

…and gives the three-line quickstart:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc   # configure (once)
cmake --build build                                    # compile
./build/hello.exe                                      # run
```

It also documents the episodes-as-branches model — each episode branch holds the full working project as of that step, so you can `git checkout ep04-menus-library` and build it — and points at `COMMANDS.md` for everything else.

`COMMANDS.md` — the command reference

`COMMANDS.md` opens by restating the core idea in one line:

> The `cmake` chain is: CMake *generates* build files → the *generator* (Ninja) compiles them.

Here's the reference, section by section. This is the project's **enduring** command list — a few sections (tests) describe tools we don't wire up until Chapter 6, so treat those as forward references you'll grow into, not commands to run at ep05.

0. Once per shell — put the toolchain on PATH. gcc needs its siblings `as` / `ld`, and CMake needs `ninja`. Add it to `~/.bashrc` to avoid retyping.

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
```

1. Configure (generate the build system) — occasionally. Needed the first time or after `rm -rf build`. The compiler + generator are locked into the cache once set.

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
```

Flag	Meaning
`-S .`	source root (where the top `CMakeLists.txt` is)
`-B build`	output dir for generated build files
`-G Ninja`	the generator (which build tool to emit files for)
`-D CMAKE_C_COMPILER=gcc`	the C compiler to use

2. Build. `--target` narrows the build to one thing:

```
cmake --build build                      # build everything
cmake --build build --target hello       # build only the app
cmake --build build --target test_math   # build only one test exe
```

3. Run the app.

```
./build/hello.exe
```

4. Tests (from inside `build/`). *(These arrive in Chapter 6 — CTest is the test runner CMake generates for you.)*

```
cd build
ctest                       # run all tests, one line each
ctest --output-on-failure   # print output of tests that FAIL
ctest -V                    # verbose: all output, even passing
ctest -N                    # list tests without running
ctest -R math               # run only tests matching "math"
ctest --rerun-failed        # re-run last failures
```

5. Run a test exe directly (raw cmocka output).

```
./build/tests/test_math.exe
```

6. Clean. Know the difference between these two:

```
cmake --build build --target clean   # remove build outputs, keep config
rm -rf build                         # nuke everything, forces reconfigure
```

The everyday inner loop, once tests exist, is one line:

```
cmake --build build && (cd build && ctest --output-on-failure)
```

And the section that motivated this whole chapter — **changing the compiler / generator** — restates the cache lock and the `rm -rf build` dance, and notes that with the Visual Studio generator the exe lands in `build/Debug/hello.exe` instead of `build/hello.exe`.

Build and run

Behavior is identical to Chapter 4 — same graph, same output. Configure, build, run:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

```
Hello, World!
2 + 3 = 5

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

What's autogenerated

Here's where the "CMake generates" idea becomes concrete. The `build/` directory is *entirely* machine-written — it's in `.gitignore` and you never edit it by hand. **What** gets written there depends on the generator:

Generator	CMake writes into `build/`	Compiled by
`-G Ninja`	`build.ninja` (plus `CMakeCache.txt`, `CMakeFiles/`)	Ninja
`-G "Visual Studio 17 2022"`	`hello.sln`, `hello.vcxproj`, …	MSBuild

Same `CMakeLists.txt`, same source files — different generated build files. That's the generator doing its one job. With Ninja you get a single `build/build.ninja`; with Visual Studio you get a solution and project files you could open in the IDE. Either way, `CMakeCache.txt` is where the locked-in compiler and generator live.

Gotchas

- **Running `cmake` from a subfolder.** The `-S .` / `-B build` paths are relative to your current directory. Run the commands from the **project root** (as `COMMANDS.md` says up top), or you'll generate a `build/` in the wrong place and confuse yourself. Always `cd` back to the root first.
- **The cache remembers a broken or old compiler.** If configuring failed (e.g. the `cannot execute 'as'` error) or you want to *switch* compiler or generator, fixing PATH or changing the `-D` / `-G` flag is **not** enough — CMake reuses the cached choice. You must `rm -rf build` and reconfigure so CMake re-evaluates the toolchain from scratch.
- **Visual Studio puts the exe under `Debug/`, Ninja does not.** With `-G "Visual Studio 17 2022"` your program is at `build/Debug/hello.exe` because MSVC is multi-config. With Ninja (single-config) it's plain `build/hello.exe`. If `./build/hello.exe` says "not found," check which generator you configured with.

Recap

- CMake is a **generator**: it does not compile — it writes build files for a build tool, which compiles. Hence the two-step configure-then-build flow.
- `-G` picks the generator. One `CMakeLists.txt` can target **Ninja** (`build.ninja`, fast, single-config, exe at `build/hello.exe`) or **Visual Studio** (`.sln` / `.vcxproj`, MSBuild, exe at `build/Debug/hello.exe`).

- The **compiler and generator are cached** at first configure. To change them, `rm -rf build` and reconfigure — new flags alone are ignored.
- The toolchain must be **on your PATH** (`export PATH="/c/MinGW/w64devkit/bin:$PATH"`); otherwise gcc can't find `as` / `ld` and you get `cannot execute 'as'` . Fix PATH, add it to `~/.bashrc` , and `rm -rf build` to clear the "broken compiler" cache.
- This episode added `README.md` (project overview) and `COMMANDS.md` (the full command reference), and formally adopted **gcc + Ninja**. No build graph changed.

Get this episode

```
git checkout ep05-generators-commands
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 6 — Unit Tests with cmocka + CTest

> Add a `tests/` folder with two cmocka test programs, register them with CTest, and learn the crucial difference between the test *framework* and the test *runner*. **Branch:** `ep06-cmocka-ctest`

What you'll build

Up to now the project has been code you run. In this chapter we add code that *checks the other code* — automated unit tests. We create a new `tests/` folder holding two small test programs:

- `tests/test_math.c` — real assertions against `add()` from our `math_utils` module: does `add(2, 3)` really return `5`?
- `tests/test_menus.c` — *smoke tests* for the `menus` library: the menu functions only print and return nothing, so there's nothing to assert; we just call them and pass if they don't crash.

On the CMake side you'll add three lines to the top `CMakeLists.txt` (`set(CMAKE_EXPORT_COMPILE_COMMANDS ON)`, `enable_testing()`, `add_subdirectory(tests)`), write a `tests/CMakeLists.txt` that builds each test into its own executable and registers it with CTest, and add a tiny `.vscode/settings.json` so VS Code stops complaining about `#include <cmocka.h>`.

By the end you'll be able to run `ctest` from the `build/` directory and see:

```
100% tests passed, 0 tests failed out of 2
```

Where we left off

Chapter 5 didn't change any code — it explained what a *generator* is (Ninja vs. Visual Studio) and shipped a `README.md` plus a `COMMANDS.md` command reference. The project itself was still the one we grew across Chapters 1–4: an app called `hello` built from `Src/`, its headers in `Inc/`, and a small `menus` library pulled in with `add_subdirectory(menus)`. The top `CMakeLists.txt` ended like this:

```
# Build an executable called "hello" from these source files (all under Src/).
add_executable(hello
    Src/main.c
    Src/math_utils.c
)

# Tell the compiler where to find this project's own headers (Inc/).
target_include_directories(hello PRIVATE Inc)

# Link the "menus" library into the app.
target_link_libraries(hello PRIVATE menus)
```

We have working code and a way to build it. What we've never had is a way to *prove* the code is correct — and to keep proving it as the project grows. That's what testing is for.

The concept

The single most important idea in this chapter is that **two separate tools** are at work, and beginners constantly confuse them. Keep them apart in your head:

cmocka is the test FRAMEWORK. It lives *inside* each test `.c` file. It gives you the machinery to write a test: assertion macros like `assert_int_equal`, a way to declare a test function (`cmocka_unit_test`), and a runner that executes a group of them (`cmocka_run_group_tests`). When you compile a test file, cmocka gets baked into the resulting `.exe`. Running that `.exe` executes the tests and reports the result through its **exit code**: `0` means every assertion passed, non-zero means at least one failed.

CTest is the test RUNNER that ships with CMake — you already have it. CTest knows *nothing* about cmocka. It doesn't understand assertions, it can't see individual test functions, it has never heard the word `assert_int_equal`. All it does is launch each program you registered with `add_test(...)`, wait for it to finish, and check the exit code: `0` = pass, anything else = fail. Then it tallies the results.

An analogy that sticks:

> **cmocka is the exam** — the actual questions a student answers inside the test. **CTest is the teacher** — who hands out the exams, collects them, and tallies who passed. The teacher never reads the questions; they only look at the final grade (the exit code).

This split explains a number that surprises everyone the first time. Each of our two test executables runs **3 cmocka unit tests** inside it. But CTest reports only **2 tests** — because CTest sees two *executables*, not six *functions*. The three assertions inside `test_math.exe` are invisible to CTest; it only sees "`test_math.exe` exited with code 0, so `math_tests` passed." Hold onto that 2-vs-3 distinction — we'll see it play out concretely when we run things.

Step by step

1. Write `test_math.c` — real assertions

Here is the complete math test. Read the top four includes carefully; they are not optional decoration.

```c
/* These four includes are required by cmocka, in this order, before cmocka.h. */
#include <stdarg.h>
#include <stddef.h>
#include <setjmp.h>
#include <cmocka.h>

#include "math_utils.h"

/* Each test takes (void **state) and uses cmocka assert_* macros. */
static void test_add_positive(void **state)
{
    (void)state;                    /* unused */
    assert_int_equal(add(2, 3), 5);
}

static void test_add_negative(void **state)
{
    (void)state;
    assert_int_equal(add(-2, -3), -5);
}

static void test_add_zero(void **state)
{
    (void)state;
    assert_int_equal(add(0, 0), 0);
}

int main(void)
{
    const struct CMUnitTest tests[] = {
        cmocka_unit_test(test_add_positive),
        cmocka_unit_test(test_add_negative),
        cmocka_unit_test(test_add_zero),
    };
    return cmocka_run_group_tests(tests, NULL, NULL);
}
```

The anatomy of a cmocka test:

- **The four includes, in that exact order, before `<cmocka.h>`.** cmocka's header uses types and macros defined by `stdarg.h` (`va_list`), `stddef.h` (`size_t`), and `setjmp.h` (`jmp_buf` — that's how cmocka jumps out of a test when an assertion fails). If you reorder them or drop one, you get cryptic compile errors *inside* `cmocka.h`. Treat the block as a fixed incantation.
- **Each test is a `static void` function taking `void **state`.** The `state` parameter is cmocka's per-test scratch pointer (for setup/teardown fixtures); we don't use it, so every test starts with `(void)state;` to silence the "unused parameter" warning.

- `assert_int_equal(actual, expected)` is the assertion. If the two ints differ, cmocka records a failure, prints where it happened, and longjmps out of the function. Pass means silence.
- `main` **builds an array of** `CMUnitTest`, wrapping each function with `cmocka_unit_test(...)`, and hands it to `cmocka_run_group_tests(tests, NULL, NULL)`. The two `NULL`s are group-level setup and teardown callbacks (none here). The return value of `cmocka_run_group_tests` becomes the program's exit code — which is exactly what CTest will read.

That's **3 unit tests** in **1 executable**.

2. Write `test_menus.c` — smoke tests

The `menus` functions (`print_main_menu`, `print_settings_menu`) return `void` and only print to stdout. There's no return value to check, so a real assertion is impossible. Instead we write **smoke tests**: call the function and treat "it didn't crash" as a pass.

```c
/* Required by cmocka, in this order, before cmocka.h. */
#include <stdarg.h>
#include <stddef.h>
#include <setjmp.h>
#include <cmocka.h>

#include <stdio.h>
#include "main_menu.h"
#include "settings_menu.h"

/*
 * These are "smoke tests": the menu functions return void and only print,
 * so there is no value to assert. We just call them and let the test pass
 * if they run without crashing. (Real assertions come once a function
 * actually returns data we can check.)
 */
static void test_main_menu_runs(void **state)
{
    (void)state;
    print_main_menu();
}

static void test_settings_menu_runs(void **state)
{
    (void)state;
    print_settings_menu();
}

/*
 * Prints every menu one after another so you can visually confirm the whole
 * output looks right. Run the exe directly to see it (ctest hides the output
 * of passing tests -- use `ctest -V` if you want it through ctest).
 */
static void test_print_all_menus(void **state)
{
    (void)state;
    printf("\n----- ALL MENUS -----\n");
    print_main_menu();
    printf("\n");
    print_settings_menu();
    printf("---------------------\n");
}

int main(void)
{
    const struct CMUnitTest tests[] = {
```

```
        cmocka_unit_test(test_main_menu_runs),
        cmocka_unit_test(test_settings_menu_runs),
        cmocka_unit_test(test_print_all_menus),
    };
    return cmocka_run_group_tests(tests, NULL, NULL);
}
```

Same skeleton as the math test — the four includes, the `void **state` functions, the `main` that runs a group. The difference is that no function calls an `assert_*` macro; the tests pass simply by returning. A smoke test is a legitimate, useful thing: it catches a function that segfaults, hits an infinite loop, or fails to link. It just can't check *correctness of output* — for that you need a function that returns data. (We'll get there in later chapters when we start mocking.)

That's another **3 unit tests** in a **2nd executable**.

3. The reusable 5-step pattern

Every test target in `tests/CMakeLists.txt` follows the same recipe. Learn it once and you can add a test for any module:

1. `add_executable` — a test is just a program with its own `main`.
2. **Add the cmocka include dir** so `#include <cmocka.h>` resolves.
3. **Link the code under test** + `libcmocka` — either compile the source straight in, or link the library that already contains it.
4. **POST_BUILD copy** `cmocka.dll` next to the `.exe` so it can launch.
5. `add_test` — register the exe with CTest.

The CMake, explained

The top `CMakeLists.txt`

Three additions turn testing on. First, near the top, right after `project(...)`:

```
# Write build/compile_commands.json listing the exact compile flags for every
# source file. VS Code's C/C++ extension reads it so IntelliSense knows the
# include paths (stdint.h, cmocka.h, ...). Ninja/Makefile generators only.
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
```

Then, at the very end of the file:

```
# Turn on CTest support, then pull in the tests folder.
enable_testing()
add_subdirectory(tests)
```

- `enable_testing()` switches on CTest for the project. It must appear (in the top-level `CMakeLists.txt`) *before* any `add_test()` call runs, which is why it comes before `add_subdirectory(tests)`. Without it, your `add_test` calls are silently ignored and `ctest` finds nothing to run.
- `add_subdirectory(tests)` tells CMake to descend into `tests/` and process its `CMakeLists.txt` — exactly like `add_subdirectory(menus)` did for the library back in Chapter 4.

- `CMAKE_EXPORT_COMPILE_COMMANDS` we'll cover in its own section below.

`tests/CMakeLists.txt`, line by line

Here's the whole file. It's the first CMake file in this book that does something genuinely new, so we'll walk every block.

```
# --- Unit tests (cmocka) ---

# Where cmocka is installed on this machine (a mingw/gcc build).
set(CMOCKA_DIR "C:/MinGW/cmocka")

# --- math module test ---
# For now we compile math_utils.c straight into the test (simplest). Once
# math_utils becomes its own library (a later episode) we'll link that instead.
add_executable(test_math
    test_math.c
    ${CMAKE_SOURCE_DIR}/Src/math_utils.c
)

target_include_directories(test_math PRIVATE
    ${CMAKE_SOURCE_DIR}/Inc      # so it finds math_utils.h
    ${CMOCKA_DIR}/include        # so it finds cmocka.h
)

# Link the cmocka library.
target_link_libraries(test_math PRIVATE ${CMOCKA_DIR}/lib/libcmocka.dll.a)

# cmocka is a DLL: copy it next to the test .exe so the test runs standalone.
add_custom_command(TARGET test_math POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
            ${CMOCKA_DIR}/bin/cmocka.dll $<TARGET_FILE_DIR:test_math>
)

# Register the exe with CTest so "ctest" runs it.
add_test(NAME math_tests COMMAND test_math)



# --- menus module test ---
# menus is already a library target, so we just link it (no source files here).
add_executable(test_menus test_menus.c)

target_include_directories(test_menus PRIVATE ${CMOCKA_DIR}/include)

# Link the menus library (brings its Inc/ headers along, since they're PUBLIC)
# and cmocka.
target_link_libraries(test_menus PRIVATE menus ${CMOCKA_DIR}/lib/libcmocka.dll.a)

add_custom_command(TARGET test_menus POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
            ${CMOCKA_DIR}/bin/cmocka.dll $<TARGET_FILE_DIR:test_menus>
)
```

```
add_test(NAME menus_tests COMMAND test_menus)
```

`set(CMOCKA_DIR "C:/MinGW/cmocka")` — cmocka isn't part of CMake or the compiler; it's a third-party library installed somewhere on this machine. We capture its root in a variable once so the paths below read cleanly. On your machine this path may differ — that's the one line you'd edit. (Forward slashes work fine in CMake even on Windows; prefer them over backslashes.)

`add_executable(test_math test_math.c ${CMAKE_SOURCE_DIR}/Src/math_utils.c)` — this is step 1 and part of step 3 at once. A test *is* a program, so we build one. Notice we compile the module under test, `math_utils.c`, **straight into the test executable**. `${CMAKE_SOURCE_DIR}` is the top of the project (where the root `CMakeLists.txt` lives), so this reaches back up to `Src/math_utils.c` from inside `tests/`. Compiling the source in directly is the simplest thing that works. The comment notes this changes later — in **Chapter 10**, once `math_utils` becomes its own library target, we'll *link* that library instead of re-listing its source here. (The committed comment says "a later episode"; that later episode is Chapter 10.)

`target_include_directories(test_math PRIVATE ${CMAKE_SOURCE_DIR}/Inc ${CMOCKA_DIR}/include)` — step 2. The test needs two search paths: our own `Inc/` (so `#include "math_utils.h"` resolves) and cmocka's `include/` (so `#include <cmocka.h>` resolves). `PRIVATE` because nothing links *against* a test executable, so these paths don't need to propagate anywhere.

`target_link_libraries(test_math PRIVATE ${CMOCKA_DIR}/lib/libcmocka.dll.a)` — the rest of step 3. `libcmocka.dll.a` is the *import library* for the cmocka DLL — a small stub the linker uses to record "this exe needs symbols from cmocka.dll at runtime." Linking it is what lets `cmocka_run_group_tests` and the `assert_*` macros resolve.

The `add_custom_command(... POST_BUILD ...)` — step 4, and the one that trips people up. Because cmocka is a **DLL** (a shared library), linking the import library is not enough: when the `.exe` runs, Windows must find `cmocka.dll` at runtime, and it looks first in the exe's own folder. So immediately after the test is built (`POST_BUILD`), we run a copy. We use CMake's own cross-platform copy, `${CMAKE_COMMAND} -E copy_if_different`, rather than a shell `copy`, so it works on any OS. `copy_if_different` skips the copy when the DLL is already there and unchanged (no needless work on rebuilds). The destination, `$<TARGET_FILE_DIR:test_math>`, is a *generator expression* — the same kind you met in Chapter 5 — that expands at build time to "the directory the `test_math` executable lands in" (here, `build/tests/`). We don't hard-code the path because it depends on the generator and configuration.

`add_test(NAME math_tests COMMAND test_math)` — step 5. This is the whole bridge to CTest: give the registered test a name (`math_tests`) and the command to run (the `test_math` target). From now on, `ctest` will launch this exe and grade it by its exit code. Note the CTest name (`math_tests`) and the target name (`test_math`) are deliberately different so we can talk about them separately.

The `test_menus` block is the same five steps with one instructive change: we **don't** recompile any source. `menus` is already a library target (from `add_subdirectory(menus)` in Chapter 4), so we just `target_link_libraries(test_menus PRIVATE menus ...)`. Because `menus` marked its `Inc/` directory `PUBLIC`, linking the library automatically drags its header search path along — which is why `test_menus` can `#include "main_menu.h"` without us adding `menus/Inc` to `target_include_directories`. This is the payoff of building a proper library target back in Chapter 4: consumers (including tests) get the headers for free. The only include dir we add by hand is cmocka's.

`CMAKE_EXPORT_COMPILE_COMMANDS` and `.vscode/settings.json`

Here's a problem you'll hit the moment you open `test_math.c` in VS Code: a red squiggle under `#include <cmocka.h>`, and IntelliSense complaining it can't find the file. The code *compiles fine* — the compiler knows the include paths because

CMake passed them. But the **editor** is a separate program that has no idea what flags CMake used. It's guessing, and it guesses wrong for a library installed off in `C:/MinGW/cmocka/include`.

The fix is two pieces:

1. `set(CMAKE_EXPORT_COMPILE_COMMANDS ON)` tells CMake to write a file called `compile_commands.json` into `build/`. This is a machine-readable list of *every* source file in the project and the *exact* compiler command line used to build it — every `-I` include path, every flag. It's the ground truth of how each file is actually compiled. (This feature only works with the Ninja and Makefile generators — which is what we use — not the Visual Studio generator.)

2. `.vscode/settings.json` points the C/C++ extension at that file:

```
{
  "C_Cpp.default.compileCommands": "${workspaceFolder}/build/compile_commands.json"
}
```

Now IntelliSense reads the same include paths the compiler used, the squiggle disappears, and go-to-definition works into `cmocka.h`. The editor and the build finally agree, because they're reading from one source of truth. Note this file only exists after you've configured the project at least once (that's when CMake writes it), so if the squiggles persist, run the `cmake -S . -B build ...` step and reload VS Code.

Build and run

Configure and build exactly as before — the new test targets get built alongside `hello`:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
```

There are **two ways** to run the tests, and seeing both makes the cmocka-vs-CTest split concrete.

Way 1 — run a test exe directly. This shows you cmocka's own output, the exam paper itself:

```
$ ./build/tests/test_math.exe
[==========] Running 3 test(s).
[ RUN      ] test_add_positive
[       OK ] test_add_positive
[ RUN      ] test_add_negative
[       OK ] test_add_negative
[ RUN      ] test_add_zero
[       OK ] test_add_zero
[==========] 3 test(s) run.
[  PASSED  ] 3 test(s).
```

That `[ RUN ]` / `[ OK ]` / `[ PASSED ]` banner is pure cmocka — CTest is not involved at all here. Notice it says **3 test(s)**: the three functions we wrote.

Way 2 — run everything through CTest. CTest must be run from *inside* the `build/` directory (that's where it finds the test registry CMake generated):

```
$ cd build
$ ctest
Test project C:/Users/tal.g/.../cmake-course/build
    Start 1: math_tests
1/2 Test #1: math_tests .......................   Passed    0.02 sec
    Start 2: menus_tests
2/2 Test #2: menus_tests ......................   Passed    0.01 sec

100% tests passed, 0 tests failed out of 2

Total Test time (real) =   0.04 sec
```

Here's the 2-vs-3 payoff. CTest says **out of 2** — it counts the two things we registered with `add_test` (`math_tests`, `menus_tests`), one per executable. The *three* cmocka tests inside each exe are invisible to it; CTest only ever saw two programs and two exit codes. cmocka counts functions; CTest counts programs.

And notice CTest printed **none** of the cmocka `[ RUN ]/[ OK ]` output above. **CTest hides the output of passing tests** — a clean run is meant to be quiet. If you want to see it, either run the exe directly (Way 1) or ask CTest for it:

```
ctest -V                    # verbose: show all output, even for passing tests
ctest --output-on-failure   # show output only for tests that FAIL
```

`test_menus`'s `test_print_all_menus` prints all the menus — you'll only see that through `ctest -V` or by running `./build/tests/test_menus.exe` directly.

What's autogenerated

Everything below lands in `build/` and is disposable — never commit it. New this chapter:

- **`build/compile_commands.json`** — written because we set `CMAKE_EXPORT_COMPILE_COMMANDS ON`. The per-file compiler command database that feeds VS Code IntelliSense. Regenerated on every configure.
- **`build/tests/test_math.exe`** **and** **`build/tests/test_menus.exe`** — the two test programs, built like any other executable.
- **`build/tests/cmocka.dll`** — the copy our `POST_BUILD` custom command drops next to each test exe so it can launch. You didn't put it there; CMake did, every build.

The `.vscode/settings.json` and the `tests/` sources, by contrast, are files *you* wrote and *do* get committed — they're inputs, not outputs.

Gotchas

- **The four cmocka includes must come before `<cmocka.h>`, in this order:** `<stdarg.h>`, `<stddef.h>`, `<setjmp.h>`, then `<cmocka.h>`. `cmocka.h` depends on types those three headers define. Reorder them, drop one, or put your own headers first, and you get baffling compile errors pointing *inside* `cmocka.h`, not at your code.

- **No `cmocka.dll` copy = the test won't even launch.** Skip the `POST_BUILD` custom command and the build succeeds, but running the exe fails immediately with a "cmocka.dll not found" error (or a silent non-launch) before a single test runs. The DLL must sit next to the `.exe`.
- **`ctest` must be run from the `build/` directory.** Run it from the project root and it prints something like "No tests were found" — the test registry lives under `build/`, and that's where CTest looks.
- **A void-returning print function can only be smoke-tested.** `test_menus` has no real assertions because `print_main_menu()` returns nothing to check. That's a limitation of the code under test, not a mistake — real assertions require a function that hands back a value. Don't fake an assertion just to have one.
- **`enable_testing()` must come before `add_test`.** It's in the top-level `CMakeLists.txt` before `add_subdirectory(tests)` for exactly this reason. Put the `add_test` calls first and CTest quietly registers nothing.
- **`CMAKE_EXPORT_COMPILE_COMMANDS` needs a re-configure to take effect**, and only the Ninja/Makefile generators write the file. If VS Code still squiggles, make sure `build/compile_commands.json` actually exists, then reload the window.

Recap

- **cmocka is the framework**: it lives inside each test `.c`, provides the four required includes, `assert_int_equal`, `cmocka_unit_test`, and `cmocka_run_group_tests`, and reports pass/fail through the program's **exit code**.
- **CTest is the runner** that ships with CMake: it knows nothing about cmocka — it just launches each exe registered with `add_test` and checks the exit code (`0` = pass). *cmocka is the exam; CTest is the teacher.*
- That's why our two exes hold **3 cmocka tests each** but `ctest` reports **2 tests** — CTest counts executables, cmocka counts functions.
- Enabling tests is three lines in the top `CMakeLists.txt`: `set(CMAKE_EXPORT_COMPILE_COMMANDS ON)`, `enable_testing()` (before any `add_test`), and `add_subdirectory(tests)`.
- Every test target follows the same **5-step pattern**: `add_executable` → add the cmocka include dir → link the code-under-test + `libcmocka` → `POST_BUILD` copy `cmocka.dll` → `add_test`.
- `test_math` compiles `math_utils.c` **straight in** (simplest for now; becomes a library link in Chapter 10); `test_menus` **links the `menus` library** and inherits its `PUBLIC` headers for free.
- `CMAKE_EXPORT_COMPILE_COMMANDS` writes `build/compile_commands.json`, and `.vscode/settings.json` points the C/C++ extension at it — that's what kills the `#include <cmocka.h>` squiggle.
- Run a test exe directly to see cmocka's `[ RUN ]/[ OK ]/[ PASSED ]` output; run `ctest` from `build/` for the tally. CTest hides passing output — use `ctest -V` or `ctest --output-on-failure` to see it.

Get this episode

```
git checkout ep06-cmocka-ctest
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
cd build && ctest --output-on-failure
```

Chapter 7 — Mocking a Hardware Leaf

> Unit-test a function that reads hardware — without any hardware — by splitting the module so the test can swap out the one function that touches a register. **Branch:** `ep07-mocking-adc`

What you'll build

An `adc` module that reads a 12-bit ADC (raw counts `0..4095`) and converts the reading to volts against a 3.0 V reference. The conversion is trivial math — `(raw / 4095) * 3.0` — but you can't run it on your laptop, because reading the ADC means poking a hardware register that only exists on the microcontroller.

So the real lesson isn't the math. It's the *structure* that lets you test the math on your PC with **no hardware at all**, by **mocking** the one function that touches the chip.

Three new source files make up the module:

File	Role
`adc/Inc/adc.h`	Declares both functions: the hardware leaf and the logic.
`adc/Src/adc.c`	The **logic** — the conversion math. No hardware access.
`adc/Src/adc_hal.c`	The **real hardware read**. Linked into the app, *not* into the test.

Plus a unit test, `tests/test_adc.c`, that supplies its own fake hardware read and checks the math for four different raw values. By the end you'll have `test_adc.exe` reporting 4 passing tests, the app printing `ADC voltage = 1.500 V`, and `ctest` running all three test suites green.

Where we left off

In Chapter 6 you stood up host-side unit testing: cmocka as the test framework, `enable_testing()` + `add_test()` to register tests with CTest, and a `tests/` folder building `test_math` and `test_menus`. Running `ctest` from `build/` ran both and reported them passing.

Those tests were easy because `add()` and the menu printers are **pure**: give them inputs, check outputs, no environment involved. This chapter tackles the case that trips everyone up the first time — testing a function whose whole job is to talk to hardware that isn't there.

The concept

Here's the problem stated plainly. You want to test this (shown with the constants inlined so the math is obvious — the committed `adc.c` factors `4095.0f` and `3.0f` into named `#define`s, which you'll see in full shortly):

```
float adc_read_voltage(void)
{
    uint16_t raw = adc_read_raw();
    return ((float)raw / 4095.0f) * 3.0f;
}
```

The math is what you care about. But `adc_read_raw()` reads an ADC data register that only exists on the MCU. Compile this for your PC and call it, and there's nothing to read. You can't test the math because it's welded to hardware you don't have.

The fix is a design idea, not a CMake trick: **split the module into two translation units so there's a seam you can cut.**

- `adc.c` holds the **logic**. It calls `adc_read_raw()` but never touches a register itself.
- `adc_hal.c` holds the **real hardware read** — the actual `adc_read_raw()`.

`adc_read_raw()` is a *hardware leaf*: the lowest-level function, the one place the register access lives. Because the logic only ever reaches hardware *through* that one function, if you can replace that function you can feed the logic any value you like.

Now the key move, and it's a **link seam** — a swap that happens at *link time*, not compile time. C's linker resolves each function name (symbol) to exactly one definition. So:

- The **app** links `adc.c` + `adc_hal.c`. The only `adc_read_raw()` present is the real one, so that's what gets used.
- The **test** links `adc.c` + its own fake `adc_read_raw()` defined inside `test_adc.c`, and **deliberately does not link `adc_hal.c`**. The real one simply isn't in the build, so the linker resolves the call to the test's fake.

```
APP  build:   adc.c  +  adc_hal.c      → adc_read_raw() = real hardware read (returns 2048)
TEST build:   adc.c  +  test_adc.c     → adc_read_raw() = fake we control
              (adc_hal.c left out)
```

That swap of one symbol at link time **is** the mock. No `#ifdef`, no function pointers, no framework magic — just controlling which object files go into the link. This is the most robust way to mock a hardware leaf in C, and the whole chapter hinges on it.

Step by step

1. **Write the leaf declaration.** Put `adc_read_raw()` in `adc.h` so both the real code and the mock agree on its signature.
2. **Isolate the logic.** `adc.c` calls `adc_read_raw()` and does the conversion — nothing else.
3. **Isolate the hardware.** `adc_hal.c` holds the real `adc_read_raw()`. On a real chip it would start a conversion and read the data register; here it returns a fixed placeholder so the app links and runs.
4. **Write the fake in the test.** `test_adc.c` defines its *own* `adc_read_raw()` — a "settable fake" backed by a `static` variable the tests write to.
5. **Wire the two build variants in CMake.** The app target links `adc` (both files). The `test_adc` target compiles `adc.c` only — never `adc_hal.c`.
6. **Assert with a tolerance.** The results are floats, so compare with an epsilon rather than expecting exact equality.

The leaf and the logic

`adc/Inc/adc.h` declares both functions and documents that the leaf is the thing tests replace:

```
#ifndef ADC_H
#define ADC_H

#include <stdint.h>

/*
 * Hardware leaf: returns one raw 12-bit ADC sample (0..4095).
 * Real implementation lives in adc_hal.c. In unit tests this symbol is
 * REPLACED by a mock so we can feed the logic any value we like.
 */
uint16_t adc_read_raw(void);

/*
 * Reads the ADC and converts the raw count to volts.
 * 12-bit full scale = 4095 counts, reference voltage Vref = 3.0 V.
 * Example: raw 4095 -> 3.0 V, raw 0 -> 0.0 V, raw 2048 -> ~1.5 V.
 */
float adc_read_voltage(void);

#endif /* ADC_H */
```

`adc/Src/adc.c` is the function under test. Notice it contains **no** hardware access — only a call to the leaf and the arithmetic:

```
#include "adc.h"

#define ADC_MAX_COUNT 4095.0f   /* 12-bit full scale */
#define ADC_VREF      3.0f      /* reference voltage in volts */

/*
 * The function under test. It contains NO hardware access itself -- it only
 * calls adc_read_raw() (the leaf) and does the conversion math. That split is
 * exactly what lets a test mock adc_read_raw() and check this math in isolation.
 */
float adc_read_voltage(void)
{
    uint16_t raw = adc_read_raw();
    return ((float)raw / ADC_MAX_COUNT) * ADC_VREF;
}
```

The real hardware read

`adc/Src/adc_hal.c` is the leaf's real implementation. Here it's a placeholder returning mid-scale, with a comment showing what real register access looks like:

```
#include "adc.h"

/*
 * The REAL hardware read. On a real MCU this would start a conversion and read
 * the ADC data register, e.g.:
 *
 *     ADC1->CR |= ADC_CR_ADSTART;
 *     while (!(ADC1->ISR & ADC_ISR_EOC)) { }
 *     return (uint16_t)ADC1->DR;
 *
 * There is no hardware here, so we return a fixed placeholder. This file is
 * linked into the app but NOT into the unit test -- the test supplies its own
 * adc_read_raw() mock instead.
 */
uint16_t adc_read_raw(void)
{
    return 2048u;   /* pretend the pin sits at mid-scale (~1.5 V) */
}
```

The mock: a "settable fake"

`tests/test_adc.c` defines its own `adc_read_raw()`. This is a **settable fake**: a `static` variable holds the value, the fake returns it, and each test writes the value it wants before calling the logic.

```c
/* Required by cmocka, in this order, before cmocka.h. */
#include <stdarg.h>
#include <stddef.h>
#include <setjmp.h>
#include <cmocka.h>

#include "adc.h"

/*
 * ===== THE MOCK (a "settable fake") =====
 * This is our own definition of adc_read_raw(). Because the test links adc.c
 * but NOT adc_hal.c, THIS definition is the one that gets used at link time --
 * it replaces the real hardware read. This "link seam" is the heart of mocking.
 *
 * The fake simply returns whatever the test put in g_fake_raw. So each test
 * sets the raw ADC value it wants, then checks that adc_read_voltage() does the
 * conversion math correctly -- with no hardware involved.
 *
 * (cmocka also has a will_return()/mock() mechanism, but on this prebuilt
 * cmocka.dll it corrupts the heap when mixed with our gcc build. A settable
 * fake is simpler and robust -- the recommended way to mock a hardware leaf.)
 */
static uint16_t g_fake_raw;

uint16_t adc_read_raw(void)
{
    return g_fake_raw;
}
```

Each test sets `g_fake_raw`, calls `adc_read_voltage()`, and asserts the result. The `#include` order matters to cmocka — the four system headers, in that exact order, come before `cmocka.h`.

The four cases cover the boundaries and a couple of points in between:

```
/* raw 0 -> 0.0 V */
static void test_voltage_zero(void **state)
{
    (void)state;
    g_fake_raw = 0;
    assert_float_equal(adc_read_voltage(), 0.0f, 0.001f);
}

/* raw 4095 (full scale) -> Vref = 3.0 V */
static void test_voltage_full_scale(void **state)
{
    (void)state;
    g_fake_raw = 4095;
    assert_float_equal(adc_read_voltage(), 3.0f, 0.001f);
}

/* raw 2048 (mid) -> 2048/4095*3 = ~1.5004 V */
static void test_voltage_midscale(void **state)
{
    (void)state;
    g_fake_raw = 2048;
    assert_float_equal(adc_read_voltage(), 1.5004f, 0.001f);
}

/* raw 1365 -> 1365/4095*3 = ~1.0 V */
static void test_voltage_one_volt(void **state)
{
    (void)state;
    g_fake_raw = 1365;
    assert_float_equal(adc_read_voltage(), 1.0f, 0.001f);
}

int main(void)
{
    const struct CMUnitTest tests[] = {
        cmocka_unit_test(test_voltage_zero),
        cmocka_unit_test(test_voltage_full_scale),
        cmocka_unit_test(test_voltage_midscale),
        cmocka_unit_test(test_voltage_one_volt),
    };
    return cmocka_run_group_tests(tests, NULL, NULL);
}
```

Notice `test_voltage_midscale` expects `1.5004f`, not `1.5f`. That's the real arithmetic: `2048 / 4095 * 3 = 1.50037…`. The tolerance argument (`0.001f`) is what lets the assertion pass anyway — more on that in Gotchas.

The CMake, explained

Three pieces change. First, the new module gets its own `adc/CMakeLists.txt`. The library is the logic **plus** the real hardware read — that's the "app" variant:

```
# --- The "adc" module ---
# Library = the conversion logic (adc.c) + the real hardware read (adc_hal.c).
# The app links both; the unit test links only adc.c and mocks the leaf.
add_library(adc
    Src/adc.c
    Src/adc_hal.c
)

target_include_directories(adc PUBLIC Inc)
```

`add_library(adc ...)` with no `STATIC` / `SHARED` keyword builds a **static** library — the default here — so you'll get `build/adc/libadc.a`. Marking the include directory `PUBLIC` means any target that links `adc` automatically inherits `adc/Inc/` on its header search path; that's why `main.c` can `#include "adc.h"` without the top-level file spelling out the path.

Second, the top-level `CMakeLists.txt` pulls the module in and links it into the app:

```
# Pull in the "adc" module (defines the "adc" library target).
add_subdirectory(adc)
```

```
# Link the "menus" library into the app. Because menus' Inc was marked PUBLIC,
# the app automatically gets menus' header search path too.
target_link_libraries(hello PRIVATE menus adc)
```

The app (`hello`) now links `adc`, which carries both `adc.c` and `adc_hal.c` — the **real** hardware read. This is the app build variant.

Third — and this is where the mock is actually wired — `tests/CMakeLists.txt` builds `test_adc` from `test_adc.c` **plus** `adc.c` **only**:

```
# --- adc module test (demonstrates MOCKING) ---
# Compile ONLY adc.c (the logic). We deliberately do NOT include adc_hal.c,
# so the mock adc_read_raw() inside test_adc.c is the definition that links.
add_executable(test_adc
    test_adc.c
    ${CMAKE_SOURCE_DIR}/adc/Src/adc.c
)

target_include_directories(test_adc PRIVATE
    ${CMAKE_SOURCE_DIR}/adc/Inc
    ${CMOCKA_DIR}/include
)

target_link_libraries(test_adc PRIVATE ${CMOCKA_DIR}/lib/libcmocka.dll.a)

add_custom_command(TARGET test_adc POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
            ${CMOCKA_DIR}/bin/cmocka.dll $<TARGET_FILE_DIR:test_adc>
)

add_test(NAME adc_tests COMMAND test_adc)
```

Read that source list carefully — it's the whole point of the chapter. The test does **not** link the `adc` library (which would drag in `adc_hal.c`); it names `adc.c` directly and stops there. Because `adc_hal.c` is absent, the only `adc_read_raw()` in the link is the fake in `test_adc.c`. Same `adc.c`, two different companions:

Target	Sources in the link	Which `adc_read_raw()` wins
`hello` (app)	`adc.c` + `adc_hal.c` (via `libadc.a`)	real hardware read → `2048`
`test_adc` (test)	`adc.c` + `test_adc.c`	fake → `g_fake_raw`

Everything else here mirrors the cmocka setup from Chapter 6: `CMOCKA_DIR` points at the installed library, `target_include_directories` adds `cmocka.h` (and this module's `adc/Inc/`), `target_link_libraries` links `libcmocka.dll.a`, the `POST_BUILD` custom command copies `cmocka.dll` next to the `.exe` so it runs standalone, and `add_test(NAME adc_tests ...)` registers it with CTest.

Build and run

Configure, build, then run the test executable directly:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/tests/test_adc.exe
```

The test binary reports all four cases passing:

```
[==========] Running 4 test(s).
[ RUN      ] test_voltage_zero
[       OK ] test_voltage_zero
[ RUN      ] test_voltage_full_scale
[       OK ] test_voltage_full_scale
[ RUN      ] test_voltage_midscale
[       OK ] test_voltage_midscale
[ RUN      ] test_voltage_one_volt
[       OK ] test_voltage_one_volt
[==========] 4 test(s) run.
[  PASSED  ] 4 test(s).
```

Run the app and you'll see it use the **real** HAL — `adc_read_raw()` returns `2048`, and `2048 / 4095 * 3 = 1.5004`, which `%.3f` rounds to `1.500`:

```
Hello, World!
2 + 3 = 5
ADC voltage = 1.500 V

... menus ...
```

And from inside `build/`, `ctest` runs all three registered suites — the two from Chapter 6 plus the new one:

```
Test project C:/.../cmake-course/build
    Start 1: math_tests
1/3 Test #1: math_tests .......................   Passed
    Start 2: menus_tests
2/3 Test #2: menus_tests ......................   Passed
    Start 3: adc_tests
3/3 Test #3: adc_tests ........................   Passed

100% tests passed, 0 tests failed out of 3
```

Same `adc.c` conversion logic, exercised two ways: fed fake values by the test, and fed the real placeholder by the app.

What's autogenerated

Everything below is produced by the build — you never edit or commit any of it:

Path	What it is
`build/adc/libadc.a`	The static `adc` library (logic + real HAL), built from `adc/CMakeLists.txt`.
`build/tests/test_adc.exe`	The compiled test binary (from `test_adc.c` + `adc.c`, with the fake linked in).
`build/tests/cmocka.dll`	Copied next to the `.exe` by the `POST_BUILD` command so the test runs standalone.

As always, the whole `build/` tree regenerates from your source files — delete it and reconfigure and you get an identical result.

Gotchas

- **Don't compile `adc_hal.c` into the test.** This is the mistake that breaks the whole technique. If you add `adc_hal.c` to the `test_adc` sources (or link the `adc` library instead of naming `adc.c` directly), you'll have **two** definitions of `adc_read_raw()` — the real one and the fake — and the link fails with a *multiple definition* / duplicate-symbol error. The mock works *precisely because* the real leaf is left out of the link.
- **Floats aren't exact — assert with an epsilon.** `2048 / 4095 * 3` is `1.50037…`, not a clean `1.5`, and even "clean" values accumulate tiny binary-floating-point error. That's why the tests use `assert_float_equal(actual, expected, 0.001f)` — the third argument is a tolerance. A plain equality check (`assert_true(actual == 1.5f)`) would be brittle and could fail on rounding alone. Pick an epsilon that's loose enough to absorb rounding but tight enough to catch real bugs.
- **Why a settable fake instead of cmocka's `will_return()` / `mock()`?** cmocka ships a return-value mechanism (`will_return()` queues a value, `mock()` pops it inside the mock function). It's idiomatic cmocka — but with *this* prebuilt `cmocka.dll` compiled by a different toolchain than our gcc, mixing it in corrupts the heap and the test crashes with exit code `c0000374` (a Windows heap-corruption code). The settable fake sidesteps that entirely, and it's also just a cleaner pattern for a hardware leaf: one variable, one return, nothing to queue. Reach for the settable fake first; save `will_return()` for when you genuinely need per-call scripted return sequences and your cmocka build is known-good.
- **The link seam swaps whole symbols, not per-call behavior.** For the lifetime of the test process, *every* call to `adc_read_raw()` hits the fake. That's exactly what you want for a leaf, but keep it in mind: the fake is process-global state (`g_fake_raw` is `static`), so set it at the top of each test rather than assuming it carries a value from a previous one.

Recap

- To test a function that touches hardware, **split the module**: put the register access in a **hardware leaf** (`adc_hal.c`) and keep the logic (`adc.c`) hardware-free so it only reaches hardware *through* the leaf.
- **Mocking here is a link seam.** The test links `adc.c` + its own fake `adc_read_raw()` and leaves `adc_hal.c` out; the linker resolves the call to the fake. That one-symbol swap at link time *is* the mock — no framework magic required.
- Two build variants share `adc.c`: the **app** links it with the real `adc_hal.c`, the **test** links it with the fake. CMake wires this by choosing which sources go into each target.
- Use a **settable fake** (`static` variable + trivial return) for hardware leaves — it's robust and dodges the `will_return()` heap-corruption issue with this prebuilt `cmocka.dll`.
- Compare floats with a **tolerance** (`assert_float_equal(..., 0.001f)`), never exact equality.
- Compiling `adc_hal.c` into the test would give a **duplicate-symbol** link error — the mock depends on leaving it out.

Get this episode

```
git checkout ep07-mocking-adc
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/tests/test_adc.exe
```

Chapter 8 — Build Options and #ifdef

> Add a feature that only exists when you ask for it. You'll wire a CMake `option()` all the way through to a C `#ifdef`, so one build flag decides whether a whole screen gets compiled in. **Branch:** `ep08-build-options`

What you'll build

An optional **debug menu**. By default it isn't there at all — the source file that implements it doesn't even get compiled. Flip one switch at configure time and the same project now builds *with* the debug menu, and `main.c` starts calling it.

Two new files carry the feature:

File	Purpose
`menus/Inc/debug_menu.h`	Declares `print_debug_menu(void)`.
`menus/Src/debug_menu.c`	Prints the `=== Debug Menu ===` screen.

The interesting part isn't the C — it's the chain that connects a CMake decision to a C preprocessor branch. By the end you'll be able to build the project two ways from the exact same sources, and you'll understand every link in that chain (including the one link that trips up almost everyone the first time: the CMake **cache**).

Where we left off

Chapter 7 finished with a `menus` library, an `adc` library, and a `hello` app that links both. The app printed a couple of numbers and then two menu screens. The relevant tail of `Src/main.c` looked like this:

```
    print_main_menu();
    printf("\n");
    print_settings_menu();
    return 0;
```

Everything that got listed in a `CMakeLists.txt` was compiled, unconditionally. That's fine when every file is always wanted. But real firmware often has parts you only want *sometimes* — a debug screen, verbose logging, a factory-test mode — and you don't want that code in the shipping binary. This chapter adds the machinery to include code conditionally.

The concept

We want a single knob that does two things at once:

1. **Decides whether `debug_menu.c` is compiled at all.**
2. **Tells `main.c` whether it's allowed to call into the debug menu.**

CMake gives you a knob with the `option()` command, and the way that knob reaches your C code is a preprocessor **define** — the same `-DNAME` flag you may have typed on a compiler command line by hand. Here's the full chain, and it's worth reading top to bottom because the rest of the chapter is just filling in each arrow:

```
option(ENABLE_DEBUG_MENU ...)         # 1. a user-settable ON/OFF knob in CMake
        |
        ▼
if(ENABLE_DEBUG_MENU)                  # 2. CMake asks: is the knob ON?
        |
        ▼
target_compile_definitions(... PUBLIC  # 3. if so, add a define to the build
    ENABLE_DEBUG_MENU)
        |
        ▼
gcc ... -DENABLE_DEBUG_MENU ...        # 4. CMake turns that into a compiler flag
        |
        ▼
#ifdef ENABLE_DEBUG_MENU               # 5. in C, the macro is now defined -> true
```

The key command is step 3, `target_compile_definitions`. **That is the command that turns a CMake decision into a `-D` flag on the compiler.** Without it, the `option()` is just a CMake variable that your C code never hears about.

A quick note on the two forms of define:

- `target_compile_definitions(t PUBLIC ENABLE_DEBUG_MENU)` produces `-DENABLE_DEBUG_MENU`. The macro is *defined* (its value is empty), which is exactly what `#ifdef` tests for. This is what we use here — it's an on/off feature flag.
- `target_compile_definitions(t PUBLIC FOO=8)` produces `-DFOO=8`, which the compiler treats as `#define FOO 8`. Use this form when the code needs an actual *value* (a buffer size, a version number) rather than a yes/no.

Step by step

1. Get onto the episode branch and put the toolchain on your PATH (see Get this episode).
2. **Add the knob.** In the top `CMakeLists.txt`, declare `option(ENABLE_DEBUG_MENU ...)` — importantly, *before* `add_subdirectory(menus)`, because the subdirectory is the code that reads the knob.
3. **React to the knob.** In `menus/CMakeLists.txt`, an `if(ENABLE_DEBUG_MENU)` block does two things: compiles the extra source with `target_sources`, and publishes the define with `target_compile_definitions`.
4. **Guard the C.** In `Src/main.c`, wrap both the `#include "debug_menu.h"` and the `print_debug_menu()` call in `#ifdef ENABLE_DEBUG_MENU ... #endif`.
5. **Configure and build twice** — once with the default (OFF) and once with the knob flipped ON — and watch the output change.

The CMake, explained

The knob (top `CMakeLists.txt`)

Right after the `set(CMAKE_EXPORT_COMPILE_COMMANDS ON)` line and *before* the menus subdirectory is pulled in, we declare the option:

```
# A user-settable build option. Default OFF. Flip it at configure time with:
#   cmake -S . -B build -D ENABLE_DEBUG_MENU=ON
# The value is remembered in the CMake cache until you change it again.
option(ENABLE_DEBUG_MENU "Include the debug menu screen" OFF)

# Pull in the "menus" subfolder. CMake runs menus/CMakeLists.txt,
# which defines the "menus" library target.
add_subdirectory(menus)
```

`option(<name> "<help text>" <default>)` creates a cache variable that is either `ON` or `OFF`. The help text shows up in CMake's GUI tools and in `cmake -LAH`. The default here is `OFF`, so out of the box the feature is absent.

Order matters: the `option()` **must be declared before** `add_subdirectory(menus)`. CMake reads files top to bottom, so if the option came *after* the subdirectory, then when `menus/CMakeLists.txt` runs its `if(ENABLE_DEBUG_MENU)` the variable wouldn't exist yet and the test would silently be false.

Reacting to the knob (`menus/CMakeLists.txt`)

The library definition is unchanged; we just append a conditional block at the end:

```
# Only when the option is ON:
if(ENABLE_DEBUG_MENU)
    # 1) compile the extra source into this library
    target_sources(menus PRIVATE Src/debug_menu.c)

    # 2) define the preprocessor macro ENABLE_DEBUG_MENU (this is what turns
    #    -D<name> on for the compiler). PUBLIC means anything linking "menus"
    #    -- like the app -- ALSO gets the macro, so main.c's #ifdef sees it.
    target_compile_definitions(menus PUBLIC ENABLE_DEBUG_MENU)
endif()
```

Two commands, two jobs:

- **`target_sources(menus PRIVATE Src/debug_menu.c)`** adds `debug_menu.c` to the `menus` library — but *only inside the* `if`. When the option is OFF, this line never runs, so `debug_menu.c` is never handed to the compiler. The dead feature costs literally nothing: no object file, no code in the binary.
- **`target_compile_definitions(menus PUBLIC ENABLE_DEBUG_MENU)`** is the link that reaches your C code. It tells CMake to pass `-DENABLE_DEBUG_MENU` when compiling — and because it's `PUBLIC`, to pass it to consumers of `menus` too.

Why `PUBLIC` here

This is the same "usage requirements" idea you met with include directories in Chapter 4, applied to a define.

The define is attached to the `menus` target. But the code that needs to see it — the `#ifdef` — lives in `main.c`, which is part of the `hello` executable, a *different* target. How does `hello` get the define?

Because `hello` links `menus` (`target_link_libraries(hello PRIVATE menus adc)`), and the define was marked `PUBLIC`. A `PUBLIC` property is used **both** when building `menus` itself **and** when building anything that links against `menus`. So the

define propagates to `hello`, and `main.c` compiles with `-DENABLE_DEBUG_MENU` even though nobody wrote that flag in the app's own `CMakeLists.txt`.

Had we written `PRIVATE` instead, `debug_menu.c` would still compile with the macro, but `main.c` would not — its `#ifdef` would stay false, the include and the call would be dropped, and you'd get a linker complaint about an unused-but-compiled function or, more likely, simply no debug menu despite building the file. `PUBLIC` is the correct choice precisely because a *consumer* (the app) needs to make a decision based on this flag.

Guarding the C (`Src/main.c`)

The C side is deliberately dumb: it just asks "is the macro defined?"

```
#include <stdio.h>
#include "math_utils.h"
#include "main_menu.h"
#include "settings_menu.h"
#include "adc.h"

#ifdef ENABLE_DEBUG_MENU
#include "debug_menu.h"
#endif

int main(void)
{
    printf("Hello, World!\n");
    printf("2 + 3 = %d\n", add(2, 3));
    printf("ADC voltage = %.3f V\n\n", adc_read_voltage());

    print_main_menu();
    printf("\n");
    print_settings_menu();

#ifdef ENABLE_DEBUG_MENU
    printf("\n");
    print_debug_menu();
#endif
    return 0;
}
```

Note there are **two** guarded regions, and both matter. The first guards the `#include` — without it, when the feature is off you'd be including a header for code that wasn't compiled. The second guards the actual call. If the macro is undefined, the preprocessor deletes both regions before the compiler ever sees them, so `main.c` compiles exactly as it did in Chapter 7.

Build and run

Default: the option is OFF

Configure and build the normal way. You don't pass `ENABLE_DEBUG_MENU` at all, so it takes its `option()` default of `OFF`:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Output — the app runs up through the Settings menu, and there is **no** debug menu:

```
Hello, World!
2 + 3 = 5
ADC voltage = 1.500 V

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

Flipped: the option is ON

Now turn the knob on with a `-D` at configure time, then rebuild:

```
cmake -S . -B build -D ENABLE_DEBUG_MENU=ON
cmake --build build
./build/hello.exe
```

Two things worth noticing about that first line. It reconfigures the *same* `build/` folder, and it **drops `-G Ninja` and `-D CMAKE_C_COMPILER=gcc`** — we don't need to repeat them because the cache already remembers the generator and compiler from the first configure. (That's the very cache behavior this chapter is about, working in your favor.)

Output — the same as before, plus the debug menu at the end:

```
Hello, World!
2 + 3 = 5
ADC voltage = 1.500 V

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back

=== Debug Menu ===
1) Dump registers
2) Force ADC value
3) Back
```

The `=== Debug Menu ===` block appears because `debug_menu.c` was compiled this time *and* `main.c` saw the macro and kept its call.

What's autogenerated

Remember `compile_commands.json` from earlier chapters — the file CMake writes listing the exact flags for every source file. It's a great way to *see* the define with your own eyes rather than take it on faith.

With the option **ON**, look at how the relevant files are compiled:

```
grep ENABLE_DEBUG_MENU build/compile_commands.json
```

You'll find `-DENABLE_DEBUG_MENU` on the command line for `main.c` and `debug_menu.c` — proof that the CMake `option()` really did become a compiler flag. With the option **OFF**, the flag is absent from every entry (and there's no entry for `debug_menu.c` at all, because it was never compiled). Everything in `build/` is generated and disposable, exactly as in Chapter 1 — this file just happens to be a handy window into what CMake decided.

Gotchas

- **The cache remembers your option — a plain reconfigure does NOT reset it.** This is the big one. When you run `cmake -S . -B build -D ENABLE_DEBUG_MENU=ON`, the value `ON` is written into `build/CMakeCache.txt` and it **persists**. Running `cmake -S . -B build` again later — with no `-D` — does *not* fall back to the `option()` default of `OFF`; it keeps `ON`, because the cache wins. To actually turn it back off you must do one of:
  - pass the opposite explicitly: `cmake -S . -B build -D ENABLE_DEBUG_MENU=OFF`,
  - edit `build/CMakeCache.txt` (or use a cache editor), or
  - delete the build tree and start clean: `rm -rf build`.

If you're ever unsure what the cache currently holds, list it:

```
cmake -LAH build
```

`-L` lists cache variables, `-A` includes advanced ones, `-H` shows each option's help string — so you'll see `ENABLE_DEBUG_MENU:BOOL=ON` (or `OFF`) with its description right there.

- **Declare `option()` before the subdirectory that uses it.** CMake executes top to bottom. If `add_subdirectory(menus)` runs before the `option()` line, the `if(ENABLE_DEBUG_MENU)` inside evaluates an undefined variable (false) and your feature silently never turns on.

- **`#ifdef` vs `#if`.** Here the macro is a bare on/off flag — when it's on, `-DENABLE_DEBUG_MENU` defines it with *no value*. `#ifdef NAME` tests "is this macro defined at all?", which is exactly right for a flag. Don't reach for `#if ENABLE_DEBUG_MENU` here: `#if` evaluates the macro's *value* as a number, and a valueless macro would break it. `#if` is for defines that carry a number (Chapter 9's `#cmakedefine01`, which forces the macro to be exactly `0` or `1` so `#if` is safe). For a simple present/absent switch like this one, `#ifdef` is the tool.

Recap

- `option(NAME "help" OFF)` creates a user-settable ON/OFF knob, defaulting off.
- The chain is `option()` → `if()` → `target_compile_definitions()` → `-DNAME` on the compiler → `#ifdef NAME` becomes true in C.
- `target_compile_definitions` is *the* command that turns a CMake decision into a `-D` flag. Use `NAME` for a flag (`-DNAME`) or `NAME=value` for a value (`-DNAME=value` → `#define NAME value`).
- Put both the extra `target_sources` and the define inside `if(ENABLE_DEBUG_MENU)`, so the feature costs nothing when off.
- `PUBLIC` makes the define propagate to consumers: `menus` sets it, and `hello` sees it because `hello` links `menus` — the same usage-requirements idea as include dirs.
- Flip it with `-D ENABLE_DEBUG_MENU=ON`. The value lives in `build/CMakeCache.txt` and **persists** until you set it otherwise or wipe `build/`. Inspect it with `cmake -LAH build`.
- Declare the `option()` *before* the subdirectory that reads it.

Get this episode

```
git checkout ep08-build-options
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
# then try it ON:
cmake -S . -B build -D ENABLE_DEBUG_MENU=ON
cmake --build build
./build/hello.exe
```

Chapter 9 — A Generated Config Header

> Replace a pile of scattered `-D` flags with **one generated header** that CMake fills in from your build options — using `configure_file()`, an INTERFACE library, and the three `#cmakedefine` syntaxes. **Branch:** `ep09-config-header`

What you'll build

The same app as before — banner, greeting, an ADC voltage, two menus — but the five build-time knobs no longer travel as loose `-D` compiler flags. Instead you write **one template**, `config/app_config.h.in`, and CMake *generates* a real header, `build/generated/app_config.h`, with every flag filled in. Your C code just does `#include "app_config.h"`.

```
cmake-course/
├── CMakeLists.txt              # declares 5 flags + configure_file() + app_config target
├── config/
│   └── app_config.h.in         # NEW: the template you write
├── Src/
│   └── main.c                  # #include "app_config.h"; uses banner/verbose/debug flags
├── adc/
│   ├── CMakeLists.txt          # links app_config PRIVATE
│   └── Src/adc.c               # uses ADC_VREF_MV instead of a hardcoded 3.0f
├── menus/
│   └── CMakeLists.txt          # keeps the conditional source, drops the -D macro
└── build/
    └── generated/
        └── app_config.h        # GENERATED at configure time — never edit this
```

The five flags we manage this way:

Flag	Kind
`ENABLE_DEBUG_MENU`	on/off
`ENABLE_STARTUP_BANNER`	on/off
`VERBOSE_LOGGING`	0/1
`ADC_VREF_MV`	integer value
`APP_VERSION`	string value (from the project version)

Where we left off

In Chapter 8 we introduced `option()` and drove one `#ifdef` by handing the compiler a macro directly. In `menus/CMakeLists.txt` we did both jobs at once:

```
if(ENABLE_DEBUG_MENU)
    # 1) compile the extra source into this library
    target_sources(menus PRIVATE Src/debug_menu.c)

    # 2) define the preprocessor macro ENABLE_DEBUG_MENU
    target_compile_definitions(menus PUBLIC ENABLE_DEBUG_MENU)
endif()
```

And the ADC's reference voltage was frozen in source:

```
#define ADC_MAX_COUNT 4095.0f   /* 12-bit full scale */
#define ADC_VREF      3.0f      /* reference voltage in volts */
```

That works for *one* flag. But the moment you have five — some on/off, some numeric, some strings — the `target_compile_definitions(... -D…)` approach gets noisy: every flag needs its own line, the values are stringly-typed, and there's no single place a reader can look to see "what is this build configured as?" This chapter fixes that.

The concept

Why not just keep adding `-D` flags?

Each `-D` flag is invisible until you dig through the build system. Ten of them scattered across three `CMakeLists.txt` files is unmanageable, and numeric or string values (`-D ADC_VREF_MV=3300` , `-D APP_VERSION=\"1.2.0\"`) are awkward to pass and easy to get wrong.

The scalable answer is a **generated config header**. You write a *template* with placeholders, and CMake's `configure_file()` command produces a real header with the placeholders replaced by the current values. Your C code includes that header and sees ordinary `#define` s. One file to read, one file that documents the whole build configuration.

The template: three substitution syntaxes

A `configure_file()` template (`.h.in` by convention) understands three special syntaxes. All five of our flags use one of them:

- `#cmakedefine NAME` — an *on/off* macro. If the CMake variable `NAME` is true, this line becomes `#define NAME` . If it's false, it becomes a *commented-out* `/* #undef NAME */` . Test it in C with `#ifdef` .
- `#cmakedefine01 NAME` — a *0/1* macro. This line *always* becomes a `#define` , either `#define NAME 0` or `#define NAME 1` . Because it is **always defined**, you test it with `#if NAME` , **not** `#ifdef` . (More on that trap in Gotchas — it's the `#1` config-header bug.)
- `@VAR@` — a *value substitution*. The token `@VAR@` is replaced verbatim with the CMake variable's value. Great for numbers and strings.

Step by step

1. Write the template `config/app_config.h.in`

This is the one file you author by hand. It's normal C with the three special syntaxes sprinkled in:

```
#ifndef APP_CONFIG_H
#define APP_CONFIG_H

/*
 * GENERATED FILE -- do not edit the copy in build/generated/app_config.h.
 * CMake produces it from this template (config/app_config.h.in) via
 * configure_file(). Change the values through CMake options instead, e.g.
 *   cmake -S . -B build -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
 */

/* --- on/off flags ---
 * Each line becomes "#define X" when the CMake variable is ON/true,
 * or a commented-out #undef when OFF/false. Test with #ifdef / #ifndef. */
#cmakedefine ENABLE_DEBUG_MENU
#cmakedefine ENABLE_STARTUP_BANNER

/* --- 0/1 flag ---
 * Always #defined, to 0 or 1. Test with #if (NOT #ifdef, since it is
 * always defined). Handy when you want an "always has a value" flag. */
#cmakedefine01 VERBOSE_LOGGING

/* --- value substitutions ---
 * @VAR@ is replaced with the CMake variable's value verbatim. */
#define ADC_VREF_MV @ADC_VREF_MV@
#define APP_VERSION "@PROJECT_VERSION@"

#endif /* APP_CONFIG_H */
```

Note `APP_VERSION`: the placeholder is `@PROJECT_VERSION@`, and it sits *inside* the C string quotes — so the substituted result is a proper string literal like `"1.2.0"`. The quotes are yours; CMake only swaps the token.

2. Declare the flags and generate the header (top `CMakeLists.txt`)

The project's version is now part of the `project()` call — that's what feeds `@PROJECT_VERSION@`:

```
# Project name, VERSION (feeds APP_VERSION below), and language(s).
project(hello VERSION 1.2.0 LANGUAGES C)
```

Then we declare all five flags, generate the header, and expose it via an INTERFACE library:

```
# ============================================================================
#  Build configuration flags (5). Change them at configure time with -D:
#    cmake -S . -B build -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
#  Values are cached until changed again (or the build dir is deleted).
# ============================================================================
option(ENABLE_DEBUG_MENU     "Include the debug menu screen"        OFF)
option(ENABLE_STARTUP_BANNER "Print the startup banner"             ON)
option(VERBOSE_LOGGING       "Emit extra verbose logging"           OFF)
set(ADC_VREF_MV 3000 CACHE STRING "ADC reference voltage in millivolts")
# 5th flag, APP_VERSION, is derived from PROJECT_VERSION (set by project()).

# Fill in app_config.h.in -> build/generated/app_config.h with the values above.
configure_file(
    ${CMAKE_SOURCE_DIR}/config/app_config.h.in
    ${CMAKE_BINARY_DIR}/generated/app_config.h
)

# A header-only (INTERFACE) target whose only job is to hand out the include
# path of the generated header. Any target that links app_config can then just
# #include "app_config.h". This is the tidy way to distribute a generated file.
add_library(app_config INTERFACE)
target_include_directories(app_config INTERFACE ${CMAKE_BINARY_DIR}/generated)
```

3. Link `app_config` from everything that needs the header

Three targets include `app_config.h`, so three targets link `app_config`. The app links it alongside its other libraries:

```
target_link_libraries(hello PRIVATE menus adc app_config)
```

The `adc` library links it privately (only `adc.c` includes it):

```
target_link_libraries(adc PRIVATE app_config)
```

And the `test_adc` test compiles `adc.c` directly, so it needs it too:

```
target_link_libraries(test_adc PRIVATE app_config ${CMOCKA_DIR}/lib/libcmocka.dll.a)
```

4. Migrate the code to read the header

`menus/CMakeLists.txt` **keeps** the conditional compile of `debug_menu.c` (whether to compile a source is still a build-system decision) but **drops** the `target_compile_definitions` — the matching macro now comes from the generated header:

```
# Whether to COMPILE debug_menu.c is a build-system decision, so it still keys
# off the CMake option directly. (The matching #ifdef macro that main.c tests
# now comes from the generated app_config.h, not a -D flag here.)
if(ENABLE_DEBUG_MENU)
    target_sources(menus PRIVATE Src/debug_menu.c)
endif()
```

`adc/Src/adc.c` includes the generated header and uses `ADC_VREF_MV` (which is in *millivolts*, hence the `/ 1000.0f`) instead of the old hardcoded `3.0f`:

```
#include "adc.h"
#include "app_config.h"        /* generated: provides ADC_VREF_MV */

#define ADC_MAX_COUNT 4095.0f  /* 12-bit full scale */

float adc_read_voltage(void)
{
    uint16_t raw = adc_read_raw();
    return ((float)raw / ADC_MAX_COUNT) * (ADC_VREF_MV / 1000.0f);
}
```

`Src/main.c` now reads all three kinds of flag from the header — note which preprocessor test goes with which kind:

```
#include <stdio.h>
#include "app_config.h"        /* generated: all 5 build flags */
#include "math_utils.h"
#include "main_menu.h"
#include "settings_menu.h"
#include "adc.h"

#ifdef ENABLE_DEBUG_MENU
#include "debug_menu.h"
#endif

int main(void)
{
#ifdef ENABLE_STARTUP_BANNER
    printf("==============================\n");
    printf("   Hello App   v%s\n", APP_VERSION);   /* value flag (string) */
    printf("==============================\n");
#endif

    printf("Hello, World!\n");
    printf("2 + 3 = %d\n", add(2, 3));
    printf("ADC voltage = %.3f V\n", adc_read_voltage());

#if VERBOSE_LOGGING                                  /* 0/1 flag -> use #if */
    printf("[verbose] Vref = %d mV\n", ADC_VREF_MV); /* value flag (int) */
#endif
    printf("\n");

    print_main_menu();
    printf("\n");
    print_settings_menu();

#ifdef ENABLE_DEBUG_MENU                             /* on/off flag -> use #ifdef */
    printf("\n");
    print_debug_menu();
#endif
    return 0;
}
```

The CMake, explained

`configure_file(input output)`

`configure_file()` reads the template, performs the substitutions, and writes the result. Its two key arguments here are absolute paths built from CMake's own variables:

- `${CMAKE_SOURCE_DIR}/config/app_config.h.in` — the template (in your source tree, tracked in git).
- `${CMAKE_BINARY_DIR}/generated/app_config.h` — the output (in the build tree, *not* tracked in git). `configure_file` creates the `generated/` folder for you.

Keeping the output under the build directory is deliberate: it's a build artifact, so it belongs with the other generated files, not next to your source.

The `app_config` INTERFACE library

```
add_library(app_config INTERFACE)
target_include_directories(app_config INTERFACE ${CMAKE_BINARY_DIR}/generated)
```

An **INTERFACE** library compiles *no* source of its own — there's no `.c` in it. It exists purely to carry *usage requirements* to whoever links it. Here the only requirement it carries is one include directory: the folder holding the generated header.

So when `hello`, `adc`, or `test_adc` do `target_link_libraries(... app_config)`, they don't pull in any object code — they inherit the `-I.../generated` include flag. That's the whole trick: instead of repeating that include path in three places, you name it once and hand it out by linking. It's the same "PUBLIC include dirs travel with the library" idea from Chapter 3, taken to its logical end — a library that is *nothing but* an include path.

Flag reference

Here is each flag: its CMake declaration, the template line, what the *default* build generates, and how to test it in C.

Flag	CMake declaration (default)	Template syntax	Generated line (default)	Test in C
`ENABLE_DEBUG_MENU`	`option(... OFF)`	`#cmakedefine ENABLE_DEBUG_MENU`	`/* #undef ENABLE_DEBUG_MENU */`	`#ifdef ENABLE_DEBUG_MENU`
`ENABLE_STARTUP_BANNER`	`option(... ON)`	`#cmakedefine ENABLE_STARTUP_BANNER`	`#define ENABLE_STARTUP_BANNER`	`#ifdef ENABLE_STARTUP_BANNER`
`VERBOSE_LOGGING`	`option(... OFF)`	`#cmakedefine01 VERBOSE_LOGGING`	`#define VERBOSE_LOGGING 0`	`#if VERBOSE_LOGGING`
`ADC_VREF_MV`	`set(... 3000 CACHE STRING ...)`	`#define ADC_VREF_MV @ADC_VREF_MV@`	`#define ADC_VREF_MV 3000`	use as int: `ADC_VREF_MV / 1000.0f`
`APP_VERSION`	`project(hello VERSION 1.2.0)` → `PROJECT_VERSION`	`#define APP_VERSION "@PROJECT_VERSION@"`	`#define APP_VERSION "1.2.0"`	use as string: `printf("%s", APP_VERSION)`

Build and run

Configure and build the defaults:

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
```

After configuring, look at what CMake generated. `build/generated/app_config.h` is the template with every placeholder resolved (the comments are copied over verbatim — this is a faithful full listing):

```
#ifndef APP_CONFIG_H
#define APP_CONFIG_H

/*
 * GENERATED FILE -- do not edit the copy in build/generated/app_config.h.
 * CMake produces it from this template (config/app_config.h.in) via
 * configure_file(). Change the values through CMake options instead, e.g.
 *   cmake -S . -B build -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
 */

/* --- on/off flags ---
 * Each line becomes "#define X" when the CMake variable is ON/true,
 * or a commented-out #undef when OFF/false. Test with #ifdef / #ifndef. */
/* #undef ENABLE_DEBUG_MENU */
#define ENABLE_STARTUP_BANNER

/* --- 0/1 flag ---
 * Always #defined, to 0 or 1. Test with #if (NOT #ifdef, since it is
 * always defined). Handy when you want an "always has a value" flag. */
#define VERBOSE_LOGGING 0

/* --- value substitutions ---
 * @VAR@ is replaced with the CMake variable's value verbatim. */
#define ADC_VREF_MV 3000
#define APP_VERSION "1.2.0"

#endif /* APP_CONFIG_H */
```

Notice how each kind resolved: the OFF `#cmakedefine` became a commented `/* #undef ... */`, the ON one became a bare `#define`, the `#cmakedefine01` became `#define VERBOSE_LOGGING 0`, and the `@…@` tokens turned into `3000` and `"1.2.0"`.

Run the app. Banner is ON, verbose is OFF, debug menu is OFF, and Vref is 3000 mV → `2048 / 4095 * 3.0 = 1.500 V`:

```
$ ./build/hello.exe
==============================
   Hello App   v1.2.0
==============================
Hello, World!
2 + 3 = 5
ADC voltage = 1.500 V

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

Now flip several flags. Because `configure_file` runs at *configure* time, you must re-run the `cmake -S . -B build …` step (not just `cmake --build`):

```
cmake -S . -B build -D ENABLE_DEBUG_MENU=ON -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
cmake --build build
```

CMake regenerates `app_config.h` (now with `#define ENABLE_DEBUG_MENU`, `#define VERBOSE_LOGGING 1`, and `#define ADC_VREF_MV 3300`). The app changes in three ways: Vref is now 3.3 V so the ADC reads `2048 / 4095 * 3.3 = 1.650 V`, the `[verbose]` line appears right after it, and the debug menu is compiled in and printed at the end:

```
$ ./build/hello.exe
==============================
   Hello App   v1.2.0
==============================
Hello, World!
2 + 3 = 5
ADC voltage = 1.650 V
[verbose] Vref = 3300 mV

=== Main Menu ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back

=== Debug Menu ===
1) Dump registers
2) Force ADC value
3) Back
```

What's autogenerated

`build/generated/app_config.h` is a **generated source file**. You never edit it — anything you type there is overwritten the next time CMake configures. The file you own is the template, `config/app_config.h.in`; the values come from CMake variables. This is the same category of artifact as the `build/compile_commands.json` you met in Chapter 6: a file CMake writes for you into the build tree, describing the current configuration. Both live under `build/` (git-ignored) precisely because they're outputs, not inputs.

If you ever wonder what a flag *actually* resolved to, open the generated header. It is the single, honest record of how this build directory is configured.

Gotchas

- `configure_file` **runs at CONFIGURE time, not build time.** Changing a flag means re-running the `cmake -S . -B build -D …` step. A plain `cmake --build build` will *not* regenerate the header, so your flag change is silently ignored until you configure again. (CMake does re-run configure automatically if you edit the `.h.in` *template*, since it tracks that as a dependency — but a `-D` on the command line only takes effect when you pass it to a configure step.)
- `#cmakedefine` **vs** `#cmakedefine01` **— the #1 config-header bug.** A `#cmakedefine01` variable is *always* `#define`d (to `0` or `1`), so `#ifdef VERBOSE_LOGGING` is **always true** — even when it's `0`. You must use `#if VERBOSE_LOGGING`. Conversely, an on/off `#cmakedefine` produces *no* macro at all when off, so `#if ENABLE_DEBUG_MENU` would error/warn

on the undefined symbol — use `#ifdef` for those. Match the test to the syntax: `#cmakedefine` → `#ifdef`, `#cmakedefine01` → `#if`.

- **Tests can couple to config.** `tests/test_adc.c` bakes in the expectation that full-scale reads **3.0 V** (`assert_float_equal(adc_read_voltage(), 3.0f, …)`) and mid-scale reads ~1.5 V. Those numbers assume `ADC_VREF_MV = 3000`. If you configure with `-D ADC_VREF_MV=3300`, the app is correct but `test_adc` *fails*, because its hardcoded expectations no longer match. Keep `ADC_VREF_MV` at its default when running the tests, or make the test compute its expected value from `ADC_VREF_MV` instead of hardcoding it.
- **Keep the quotes for string substitutions.** `@PROJECT_VERSION@` expands to `1.2.0` (no quotes). The template writes `"@PROJECT_VERSION@"` so the result is a valid C string literal. Drop the quotes and `#define APP_VERSION 1.2.0` is a syntax error the moment you `printf("%s", APP_VERSION)`.

Recap

- Scattered `-D` flags don't scale. A **generated config header** collects the whole build configuration into one readable file.
- `configure_file(template output)` fills a `.h.in` template with CMake values at configure time.
- Three template syntaxes: `#cmakedefine` (on/off → `#define` /commented `#undef`, test with `#ifdef`), `#cmakedefine01` (always `0` / `1`, test with `#if`), and `@VAR@` (verbatim value substitution for ints and strings).
- An **INTERFACE library** (`app_config`) carries no code — just the generated header's include path — so any target that links it can `#include "app_config.h"`. Here `hello`, `adc`, and `test_adc` all link it.
- The generated `app_config.h` is a build artifact (like `compile_commands.json`): never edit it; edit the template and re-configure.

Get this episode

```
git checkout ep09-config-header
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Chapter 10 — Library-to-Library Dependencies

> Promote `math_utils` into its own library module so the app, the `menus` library, and the unit test all share one definition of `add()` — and meet **transitive linking**, the rule that makes static-library dependencies propagate automatically. **Branch:** `ep10-lib-plus-lib`

What you'll build

The program prints exactly what it did in Chapter 9. Nothing about the *output* changes here (well — almost nothing; the main menu learns to count its own options). What changes is the **shape of the project**: `math_utils` stops being a pair of loose files compiled into whatever needs it, and becomes a real library module with its own folder, its own `CMakeLists.txt`, and its own `PUBLIC` include directory.

Once it is a library, three completely different targets can all link the *same* compiled `math_utils`:

- the app (`hello`) — `main.c` calls `add(2, 3)`
- the `menus` library — `main_menu.c` calls `add(2, 1)` to count its options
- the unit test (`test_math`) — links `math_utils` instead of recompiling it

Here is the layout after this episode. The `math/` folder is new; the old flat `Inc/math_utils.h` and `Src/math_utils.c` are **gone**:

```
cmake-course/
├── CMakeLists.txt
├── config/
│   └── app_config.h.in
├── math/                 ← NEW module
│   ├── CMakeLists.txt
│   ├── Inc/
│   │   └── math_utils.h  ← moved here from top-level Inc/
│   └── Src/
│       └── math_utils.c  ← moved here from top-level Src/
├── menus/
│   ├── CMakeLists.txt
│   ├── Inc/
│   └── Src/
├── adc/
│   ├── CMakeLists.txt
│   ├── Inc/
│   └── Src/
├── Src/
│   └── main.c            ← the ONLY source the app compiles directly now
└── tests/
    ├── CMakeLists.txt
    ├── test_math.c
    ├── test_menus.c
    └── test_adc.c
```

This is the finale of the course, and it is where every idea we've built up — libraries, `PUBLIC` / `PRIVATE`, subdirectories, include directories, tests — clicks together into the pattern real projects actually use.

Where we left off

By the end of Chapter 9, the project already had two library modules — `menus` and `adc` — each in its own folder with its own `add_library` and a `PUBLIC` include directory. We had a generated config header (`app_config.h`) handed out by an `INTERFACE` target called `app_config` . And `math_utils` was... still stuck in the past.

`math_utils.h` sat in a top-level `Inc/` , `math_utils.c` in a top-level `Src/` , and the code got into the app the old-fashioned way — by listing the `.c` file directly in `add_executable` and putting `Inc/` on the app's private include path:

```
# Chapter 9's app target
add_executable(hello
    Src/main.c
    Src/math_utils.c        # ← math_utils compiled straight into the app
)
target_include_directories(hello PRIVATE Inc)   # ← so it finds math_utils.h
```

The unit test did something worse: it **recompiled** the same source into its own executable:

```
# Chapter 9's test
add_executable(test_math
    test_math.c
    ${CMAKE_SOURCE_DIR}/Src/math_utils.c    # ← second, separate compile of add()
)
```

So `add()` was compiled twice — once for the app, once for the test — from the same source. It worked, but it means the test wasn't verifying the *exact* object the app ships. And if `menus` had wanted to call `add()`, it had no clean way to get at it. `math_utils` was the last module that wasn't really a module. This chapter fixes that.

The concept

Every reusable piece becomes a library

The organizing idea of this whole course, stated once more in its final form: **if code is used by more than one thing, make it a library.** A library is a named target you build once and link wherever you need it. `menus` and `adc` already earn their keep this way. `math_utils` is used by the app, and now by `menus`, and by a test — three consumers. That is three reasons to promote it.

Promoting it is a fixed, three-line recipe you've now seen enough times to recognize:

```
add_library(math_utils Src/math_utils.c)
target_include_directories(math_utils PUBLIC Inc)
```

`PUBLIC` on the include directory means: use `Inc/` when compiling `math_utils` itself, **and** hand `Inc/` to anything that links `math_utils`, so consumers can `#include "math_utils.h"` without repeating the path. That's the same `PUBLIC` you met with `menus` in Chapter 4 — it's how a library carries its own headers to its users.

A library depending on a library

The new move this episode is one library linking another. `menus` now *uses* `math_utils`:

```
# in menus/CMakeLists.txt
target_link_libraries(menus PRIVATE math_utils)
```

Why `PRIVATE`? Because `math_utils` is an **internal implementation detail** of `menus`. Look at where `add()` is called — inside `main_menu.c`, a `.c` file. The `menus` *headers* (`main_menu.h`, `settings_menu.h`) never mention `math_utils` at all. So code that merely `#include`s a menus header has no need to know about `math_utils`. `PRIVATE` says exactly that: "`menus` needs `math_utils` to build itself, but users of `menus` don't inherit it."

Contrast with `PUBLIC`, which you'd use if `main_menu.h` had a function whose prototype mentioned a `math_utils` type — then a consumer including that header *would* need `math_utils`'s include dir, and `PUBLIC` would forward it. It doesn't here, so `PRIVATE` is correct.

Transitive linking — the heart of this chapter

Here's the part worth slowing down for. Look at the menus **test**:

```
add_executable(test_menus test_menus.c)
target_link_libraries(test_menus PRIVATE menus ${CMOCKA_DIR}/lib/libcmocka.dll.a)
```

`test_menus` links `menus`. It does **not** name `math_utils` anywhere. Yet `test_menus.c` calls `print_main_menu()`, which calls `add()` — a symbol that lives in `math_utils`. How does that link succeed?

The answer is two separate mechanisms that `PRIVATE` governs, and it's important not to blur them together:

1. The *link requirement* propagates — always, regardless of PRIVATE. `menus` is a **static library**: an archive (`libmenus.a`) of compiled `.o` files. A static library does *not* absorb the objects of the libraries it depends on — `libmenus.a` does not contain a copy of `add()`. So inside `libmenus.a`, the call to `add()` is an **unresolved symbol**. It has to be satisfied at the moment something is finally linked into an executable. CMake knows this, so when it builds the link line for `test_menus`, it automatically appends `math_utils` — even though you only wrote `menus`. This is how static-library link requirements *propagate* to whoever links the library. It happens whether the dependency was declared `PRIVATE` or `PUBLIC`, because it's a physical necessity: the symbol must resolve or the link fails.

2. The *usage requirements* do NOT propagate — because it's PRIVATE. "Usage requirements" are things like include directories and compile definitions. Because `menus` linked `math_utils` as `PRIVATE`, a consumer of `menus` does **not** inherit `math_utils`'s `PUBLIC` include directory. And indeed `test_menus.c` includes `main_menu.h` and `settings_menu.h` but never `math_utils.h` — it doesn't need to, because it never names `add()` directly. The include path stays private to `menus`. (Had the dependency been `PUBLIC`, `test_menus` *would* have gotten `math_utils/Inc` on its include path too.)

3. And `PRIVATE` still lets `menus` itself use the dependency. This is the third face of the same keyword. `PRIVATE` isn't "off" — it's "for me, not my users." So `main_menu.c` can `#include "math_utils.h"` and call `add()` and compile cleanly, because when building `menus` *itself*, math_utils's include dir and its object are both in play.

So the one-line summary to *avoid* is "PRIVATE means it doesn't propagate." Too crude. The precise version: **the link requirement propagates (static libs force it); the usage requirements (includes/defines) do not, because it's PRIVATE; and the library itself gets full use of the dependency either way.** That three-way distinction is the real lesson of this chapter.

Why the app still links `math_utils` explicitly

You'll notice the app names `math_utils` directly:

```
target_link_libraries(hello PRIVATE math_utils menus adc app_config)
```

That's not redundant with the transitive story. `main.c` calls `add(2, 3)` *itself* (`2 + 3 = 5`), so `hello` is a first-class, direct consumer of `math_utils` — it doesn't rely on `menus` to drag the symbol in. When a target uses a symbol directly, you list the library that provides it directly. Letting it arrive only transitively would be fragile and would read as an accident.

Each library carries its own include directory

One more consequence to spell out. In Chapter 9 the app had this line:

```
target_include_directories(hello PRIVATE Inc)   # so it could find math_utils.h
```

It's **gone** now. The app has *no* `target_include_directories` at all. Why? Because every header the app needs now comes from a library's `PUBLIC` include directory, forwarded automatically when the app links that library:

Header the app includes	Comes from
`math_utils.h`	`math_utils`'s `PUBLIC Inc`
`main_menu.h`, `settings_menu.h`	`menus`'s `PUBLIC Inc`
`adc.h`	`adc`'s `PUBLIC Inc`
`app_config.h` (generated)	`app_config`'s `INTERFACE` include dir

The app just links the four targets and every include path comes along for the ride. This is the payoff of `PUBLIC`: a well-formed library is *self-describing* — link it and its headers are simply there.

Step by step

1. Create the `math/` module folder and move the two files into it. No edits to the file *contents* — only their location:

- `Inc/math_utils.h` → `math/Inc/math_utils.h`
- `Src/math_utils.c` → `math/Src/math_utils.c`

The top-level `Inc/` and `Src/math_utils.c` no longer exist (the app's `Src/` now holds only `main.c`).

2. Write `math/CMakeLists.txt` — the same `add_library` + `target_include_directories(... PUBLIC Inc)` recipe you used for `menus` and `adc`.

3. In the top `CMakeLists.txt`, add `add_subdirectory(math)` **before** `add_subdirectory(menus)` (menus links math_utils, so math_utils's target must exist first), drop `Src/math_utils.c` from `add_executable(hello ...)`, delete the app's `target_include_directories`, and add `math_utils` to the app's `target_link_libraries`.

4. In `menus/CMakeLists.txt`, add `target_link_libraries(menus PRIVATE math_utils)`.

5. In `menus/Src/main_menu.c`, include `math_utils.h` and use `add()` to compute the option count.

6. In `tests/CMakeLists.txt`, change `test_math` to link `math_utils` instead of recompiling `math_utils.c`, and drop the now-unnecessary include dir.

The CMake, explained

The new module: `math/CMakeLists.txt`

```
# --- The "math_utils" module ---
# math_utils is now its own library, so ANY target can link it: the app, the
# menus library, and the unit test all share this one definition of add().
add_library(math_utils
    Src/math_utils.c
)

target_include_directories(math_utils PUBLIC Inc)
```

Paths here are relative to `math/` (the folder holding this `CMakeLists.txt`), so `Src/math_utils.c` and `Inc` point inside the module. This is byte-for-byte the same shape as `menus/CMakeLists.txt` and `adc/CMakeLists.txt` — that sameness is the point. Every module looks like every other module.

The moved files are unchanged. For reference:

`math/Inc/math_utils.h`:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/* Returns the sum of a and b. */
int add(int a, int b);

#endif /* MATH_UTILS_H */
```

`math/Src/math_utils.c`:

```
#include "math_utils.h"

int add(int a, int b)
{
    return a + b;
}
```

`menus` depends on `math_utils`

`menus/CMakeLists.txt` gains one line (shown in context):

```
# --- The "menus" module ---
# Bundle this folder's sources into a library called "menus".
add_library(menus
    Src/main_menu.c
    Src/settings_menu.c
)

# PUBLIC means: this include dir is used both when building "menus" itself
# AND automatically added to anything that links against "menus".
# So the main app can #include "main_menu.h" without repeating this path.
target_include_directories(menus PUBLIC Inc)

# menus' own code calls add() from math_utils. PRIVATE: it's an internal
# implementation detail -- code that uses menus' headers doesn't need it.
target_link_libraries(menus PRIVATE math_utils)

# Whether to COMPILE debug_menu.c is a build-system decision, so it still keys
# off the CMake option directly. (The matching #ifdef macro that main.c tests
# now comes from the generated app_config.h, not a -D flag here.)
if(ENABLE_DEBUG_MENU)
    target_sources(menus PRIVATE Src/debug_menu.c)
endif()
```

And `menus/Src/main_menu.c` now actually *uses* the dependency:

```
#include <stdio.h>
#include "main_menu.h"
#include "math_utils.h"   /* menus now depends on the math_utils library */

void print_main_menu(void)
{
    /* Demo of a library-to-library dependency: menus calls add() from
       math_utils to compute how many options it shows. */
    int option_count = add(2, 1);

    printf("=== Main Menu (%d options) ===\n", option_count);
    printf("1) Start\n");
    printf("2) Settings\n");
    printf("3) Quit\n");
}
```

`add(2, 1)` returns `3`, so the header line becomes `=== Main Menu (3 options) ===`. A tiny, contrived use — but a genuine one: the `add()` symbol truly comes from another library, and that's what makes this a real library-to-library dependency rather than a diagram of one.

The top `CMakeLists.txt`

Here is the full file for this episode. The lines that changed from Chapter 9 are called out below it:

```
# Minimum CMake version required to process this file.
cmake_minimum_required(VERSION 3.15)

# Project name, VERSION (feeds APP_VERSION below), and language(s).
project(hello VERSION 1.2.0 LANGUAGES C)

# Write build/compile_commands.json listing the exact compile flags for every
# source file. VS Code's C/C++ extension reads it so IntelliSense knows the
# include paths (stdint.h, cmocka.h, ...). Ninja/Makefile generators only.
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# ======================================================================
#  Build configuration flags (5). Change them at configure time with -D:
#    cmake -S . -B build -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
#  Values are cached until changed again (or the build dir is deleted).
# ======================================================================
option(ENABLE_DEBUG_MENU     "Include the debug menu screen"        OFF)
option(ENABLE_STARTUP_BANNER "Print the startup banner"             ON)
option(VERBOSE_LOGGING       "Emit extra verbose logging"           OFF)
set(ADC_VREF_MV 3000 CACHE STRING "ADC reference voltage in millivolts")
# 5th flag, APP_VERSION, is derived from PROJECT_VERSION (set by project()).

# Fill in app_config.h.in -> build/generated/app_config.h with the values above.
configure_file(
    ${CMAKE_SOURCE_DIR}/config/app_config.h.in
    ${CMAKE_BINARY_DIR}/generated/app_config.h
)

# A header-only (INTERFACE) target whose only job is to hand out the include
# path of the generated header. Any target that links app_config can then just
# #include "app_config.h". This is the tidy way to distribute a generated file.
add_library(app_config INTERFACE)
target_include_directories(app_config INTERFACE ${CMAKE_BINARY_DIR}/generated)

# math_utils is now a library module too. It must come before menus, which
# links it.
add_subdirectory(math)

# Pull in the "menus" subfolder. CMake runs menus/CMakeLists.txt,
# which defines the "menus" library target.
add_subdirectory(menus)

# Pull in the "adc" module (defines the "adc" library target).
add_subdirectory(adc)

# Build an executable called "hello". The app is now just main.c -- every other
```

```
# piece of code lives in a library.
add_executable(hello
    Src/main.c
)

# Link the libraries into the app. Each library brings its own PUBLIC include
# dirs, so the app needs no target_include_directories of its own anymore:
#   math_utils -> math_utils.h    menus/adc -> their headers
#   app_config -> generated app_config.h
target_link_libraries(hello PRIVATE math_utils menus adc app_config)

# Turn on CTest support, then pull in the tests folder.
enable_testing()
add_subdirectory(tests)
```

What changed:

- `add_subdirectory(math)` **appeared**, positioned *before* `add_subdirectory(menus)`. Ordering matters here: `menus/CMakeLists.txt` calls `target_link_libraries(menus PRIVATE math_utils)`, and the `math_utils` target has to already be defined when CMake processes that line. (More on this in Gotchas.)
- `Src/math_utils.c` **is gone from** `add_executable`**.** The app compiles a single file now — `Src/main.c`. Everything else is a library.
- `target_include_directories(hello PRIVATE Inc)` **is deleted.** The app owns no include directories anymore; every header arrives via a linked library's `PUBLIC` / `INTERFACE` include dir.
- `math_utils` **was added to the link line.** `target_link_libraries(hello PRIVATE math_utils menus adc app_config)` — because `main.c` calls `add()` directly.

The tests

`tests/CMakeLists.txt` for `test_math` slims down considerably. Before, it recompiled `math_utils.c` and pointed an include dir at the old top-level `Inc/`. Now it just links the library:

```
# --- math module test ---
# Now that math_utils is a library, we LINK it instead of recompiling its
# source. Cleaner, and it tests the exact same object the app uses.
add_executable(test_math test_math.c)

# cmocka.h path only -- math_utils.h comes from the math_utils PUBLIC include dir.
target_include_directories(test_math PRIVATE ${CMOCKA_DIR}/include)

# Link math_utils (the code under test) and cmocka.
target_link_libraries(test_math PRIVATE math_utils ${CMOCKA_DIR}/lib/libcmocka.dll.a)
```

Two benefits, both real:

- **The test exercises the exact same compiled object the app links.** There's now **one** compilation of `add()`, in `libmath_utils.a`, shared by app and test. Previously the test had its own separate build of the source — if a compile

flag differed, you could in principle test something subtly unlike what shipped. That gap is closed.
- **`test_math` no longer needs a path to `math_utils.h`.** It comes from `math_utils`'s `PUBLIC Inc`, forwarded on link. The only include dir the test still names is cmocka's.

`test_menus` is the transitive-linking exhibit discussed above — it links `menus`, names no `math_utils`, and still resolves `add()`:

```
# --- menus module test ---
# menus is already a library target, so we just link it (no source files here).
add_executable(test_menus test_menus.c)

target_include_directories(test_menus PRIVATE ${CMOCKA_DIR}/include)

# Link the menus library (brings its Inc/ headers along, since they're PUBLIC)
# and cmocka.
target_link_libraries(test_menus PRIVATE menus ${CMOCKA_DIR}/lib/libcmocka.dll.a)
```

(`test_adc` is unchanged from Chapter 9 — it still compiles `adc.c` on its own and mocks the hardware leaf, links `app_config`, and registers with CTest.)

Build and run

Configure and build exactly as before. From the project root:

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
```

Expected output (banner on, verbose off, debug menu off — all the defaults):

```
==============================
   Hello App   v1.2.0
==============================
Hello, World!
2 + 3 = 5
ADC voltage = 1.500 V

=== Main Menu (3 options) ===
1) Start
2) Settings
3) Quit

=== Settings ===
1) Volume
2) Brightness
3) Back
```

The only visible difference from Chapter 9 is `(3 options)` in the main-menu header — proof that `menus` really is calling `add()` from the `math_utils` library. `2 + 3 = 5` is the app's own direct call to `add()`; `ADC voltage = 1.500 V` is the `adc` library reading a mid-scale count (2048 of 4095) against the 3.0 V reference.

Now run the tests. From inside `build/`:

```
cd build && ctest
```

```
Test project C:/Users/.../cmake-course/build
    Start 1: math_tests
1/3 Test #1: math_tests .......................   Passed    0.02 sec
    Start 2: menus_tests
2/3 Test #2: menus_tests ......................   Passed    0.02 sec
    Start 3: adc_tests
3/3 Test #3: adc_tests ........................   Passed    0.02 sec

100% tests passed, 0 tests failed out of 3
```

All three suites pass. `math_tests` now runs against the shared `libmath_utils.a`, and `menus_tests` passes only *because* transitive linking pulled `math_utils` onto its link line to resolve `add()`.

What's autogenerated

The `build/` tree now contains a genuinely modular set of build products. Because each module lives in its own subdirectory, its static library is generated under a matching subfolder of `build/`:

```
build/
├── hello.exe                ← the app (compiles just Src/main.c)
├── math/
│   └── libmath_utils.a      ← NEW: the math_utils static library
├── menus/
│   └── libmenus.a           ← the menus static library
├── adc/
│   └── libadc.a             ← the adc static library
├── generated/
│   └── app_config.h         ← from configure_file()
├── tests/
│   ├── test_math.exe
│   ├── test_menus.exe
│   ├── test_adc.exe
│   └── cmocka.dll           ← copied next to the tests by a POST_BUILD step
├── compile_commands.json    ← from CMAKE_EXPORT_COMPILE_COMMANDS
├── build.ninja
└── CMakeCache.txt, CMakeFiles/, ...
```

Those `.a` files are **static libraries** — archives of compiled `.o` objects. `libmath_utils.a` holds exactly one definition of `add()`, and it's this archive that both `hello.exe` and `test_math.exe` link against. As always, everything under `build/` is generated and git-ignored; you never edit it by hand.

Gotchas

- **`add_subdirectory(math)` must come before anything that links `math_utils`.** CMake processes your `CMakeLists.txt` top to bottom. When it reaches `target_link_libraries(menus PRIVATE math_utils)` (inside the `menus` subdirectory), the `math_utils` target has to already exist. Since `menus` is pulled in by `add_subdirectory(menus)`, the `add_subdirectory(math)` line must appear *above* it — which is exactly why it's placed where it is. Put `math` after `menus` and you'll get an error like:

  ```
  Target "menus" links to target "math_utils" but the target was not found.
  ```

  (Strictly, CMake can sometimes tolerate forward references between top-level targets, but relying on that is asking for trouble — declare a module before its consumers and the ordering is never in question.)

- **Circular library dependencies are not allowed.** `menus` links `math_utils`. If you then made `math_utils` link `menus`, you'd have a cycle — A needs B needs A — and CMake will refuse it. Dependencies must form a one-way graph (a *DAG*): higher-level modules depend on lower-level ones, never the reverse. If two modules genuinely need each other, that's a design smell — usually the shared piece wants to be factored into a third, lower-level library that both depend on.

- **`math_utils.h` now lives in `math/Inc/` only.** The old top-level `Inc/` is gone, and with it the app's `target_include_directories(hello PRIVATE Inc)`. The one and only thing that puts `math_utils.h` on a consumer's include path is `math_utils`'s `PUBLIC Inc` directory, delivered when you link `math_utils`. If you ever see `fatal error:`

`math_utils.h: No such file or directory`, the question is no longer "did I add an include dir?" but "did this target link `math_utils`?" — linking the library *is* how you get its header.

- **Don't confuse "links `menus`" with "gets math_utils's headers."** Because `menus` links `math_utils` as `PRIVATE`, linking `menus` gives you `menus`'s headers but **not** `math_utils`'s. `test_menus` proves this works as intended: it resolves `add()` at link time (the link requirement propagates) yet never gets `math_utils.h` on its include path (the usage requirement doesn't). If you *wanted* `math_utils.h` visible to `menus`'s consumers, you'd declare the dependency `PUBLIC` — but here you don't, so `PRIVATE` is right.

Recap

- If code is used by more than one target, it should be a **library**. This episode promotes `math_utils` into its own module (`math/`), removing the old flat `Inc/math_utils.h` and `Src/math_utils.c`, so the app, `menus`, and the unit test all share one compiled definition of `add()`.
- Making a module is a **fixed recipe**: a folder with `add_library(...)` + `target_include_directories(... PUBLIC Inc)`, then `add_subdirectory(newmod)` and add it to a consumer's `target_link_libraries`. Every module in the project now looks identical — that sameness is how large projects stay manageable.
- A library can link another library. `menus` links `math_utils` **PRIVATE** because `math_utils` is an internal implementation detail — used in `menus`'s `.c`, never exposed by its headers.
- **Transitive linking** has three faces, and they must not be flattened into one slogan: (1) the **link requirement propagates** — because static libraries don't absorb their dependencies' objects, CMake automatically adds `math_utils` to the link line of anything that links `menus`, so `add()` resolves; (2) the **usage requirements do not propagate** under `PRIVATE` — a consumer of `menus` does not inherit `math_utils`'s include dir; (3) `menus` **itself** still enjoys full use of `math_utils` when compiling its own sources.
- Each library carries its **own `PUBLIC` include directory** to consumers, which is why the app dropped `target_include_directories` entirely — linking a library brings its headers along for free.
- The app links `math_utils` **directly** because `main.c` calls `add()` itself; a symbol you use directly should come from a library you name directly, not one you happen to get transitively.

Get this episode

```
git checkout ep10-lib-plus-lib
export PATH="/c/MinGW/w64devkit/bin:$PATH"
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
cmake --build build
./build/hello.exe
cd build && ctest
```

And that's the course. You started in Chapter 1 with a single `add_executable` and a "Hello, World!", and you've arrived at a properly modular project: several library modules that each own their sources and headers, a generated config header driven by build options, a real unit-test suite wired into CTest, and libraries that depend on other libraries with their link and usage requirements propagating correctly. That is genuinely how professional C and C++ codebases are structured — you now have the whole shape, not a toy version of it. Congratulations on finishing. When you need to look a command up again, the appendices are your quick reference: **Appendix A** collects every CMake command you met, and the later appendices cover the compiler-flag and testing recipes. Go build something.

Appendix A — Command Reference

Every command runs from the **project root** unless noted. First put the toolchain on PATH (once per shell):

```
export PATH="/c/MinGW/w64devkit/bin:$PATH"
```

Configure (generate the build system)

```
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=gcc
```

`-S .` source dir · `-B build` output dir · `-G Ninja` generator · `-D CMAKE_C_COMPILER=gcc` compiler. Needed the first time, or after `rm -rf build`. Compiler + generator are locked into the cache once set.

Build

```
cmake --build build                      # build everything
cmake --build build --target hello       # build only the app
cmake --build build --target test_math   # build only one test exe
```

Run the app

```
./build/hello.exe
```

Tests — from inside `build/`

```
cd build
ctest                       # run all tests, one line each
ctest --output-on-failure   # print output of tests that FAIL
ctest -V                    # verbose: all output, even passing
ctest -N                    # list tests without running
ctest -R math               # run only tests matching "math"
ctest --rerun-failed        # re-run last failures
```

Run a test exe directly (raw cmocka output)

```
./build/tests/test_math.exe
./build/tests/test_menus.exe
./build/tests/test_adc.exe
```

Build options (Chapters 8–9)

```
cmake -S . -B build -D ENABLE_DEBUG_MENU=ON       # turn a flag on
cmake -S . -B build -D VERBOSE_LOGGING=ON -D ADC_VREF_MV=3300
cmake -LAH build                                  # list all cached options
```

Remember: option values live in `build/CMakeCache.txt` and persist until you change them again or delete `build/`.

Clean

```
cmake --build build --target clean   # remove build outputs, keep config
rm -rf build                         # nuke everything, forces reconfigure
```

The everyday loop

```
cmake --build build && (cd build && ctest --output-on-failure)
```

Changing compiler / generator

```
rm -rf build
cmake -S . -B build -G Ninja -D CMAKE_C_COMPILER=clang    # different compiler
cmake -S . -B build -G "Visual Studio 17 2022"           # different generator
```

With the Visual Studio generator the exe is at `build/Debug/hello.exe` instead of `build/hello.exe`.

Appendix B — cmocka vs CTest

These two are constantly confused. They are different layers, and you use both.

cmocka — the test *framework*

cmocka is the **library your test code is written against**. It lives *inside* each test executable and provides:

- assertions: `assert_int_equal`, `assert_float_equal(a, b, epsilon)`, ...
- the runner machinery: `cmocka_unit_test(...)`, `cmocka_run_group_tests(...)`
- the `[ RUN ] / [ OK ] / [ PASSED ]` output you see

A cmocka test program reports its result through its **exit code**: `0` when all tests pass, non-zero when any fail.

> Reminder: the four includes must come **before** `<cmocka.h>`, in this order: `<stdarg.h>`, `<stddef.h>`, `<setjmp.h>`, then `<cmocka.h>`.

CTest — the test *runner*

CTest ships with CMake and knows **nothing** about cmocka. All it does:

1. reads every `add_test(NAME ... COMMAND ...)` you wrote in CMake,
2. launches each of those executables,
3. checks the exit code — `0` = pass, anything else = fail,
4. tallies a summary.

Those executables happen to be built with cmocka, but they could be anything that returns 0/non-zero.

How they fit together

```
ctest                        ← CMake's runner: launches exes, checks exit codes
  | runs
  ▼
test_math.exe  test_menus.exe  test_adc.exe   ← executables you built
  | built from
  ▼
test_math.c    test_menus.c    test_adc.c     ← YOUR test code, using...
  | uses
  ▼
cmocka         ← the framework: assert_*, run_group_tests, exit code
```

Analogy: cmocka is the *exam* (the questions and the grading); CTest is the *teacher* who hands out every exam, collects them, and writes "3/3 passed" on the board. The teacher doesn't grade individual questions — the exam does that and reports pass/fail.

Do you need CTest?

No. You can run the exes yourself:

```
./build/tests/test_math.exe && ./build/tests/test_menus.exe && ./build/tests/test_adc.exe
```

But CTest is worth it as tests grow — one command runs them all, plus filtering (`ctest -R math`), re-running failures (`--rerun-failed`), parallelism (`ctest -j`), and a uniform summary. One caveat: CTest **hides the output of passing tests** — use `ctest -V` , or run the exe directly, to see prints (like the menu dump in `test_menus`).

Appendix C — Files CMake Generates for You

A recurring question in this course was "which files are autogenerated?" Here's the complete picture. **Rule of thumb: everything under `build/` is generated and must never be committed or hand-edited.** That's why the very first commit included a `.gitignore` with `/build/`.

Everything in `build/`

Path	What it is	Who makes it
`build/CMakeCache.txt`	Cached settings: compiler path, generator, every `option()` / `-D` value	`cmake` configure
`build/CMakeFiles/`	Internal bookkeeping — compiler checks, dependency data	`cmake` configure
`build/build.ninja`, `build/rules.ninja`	The Ninja build description CMake wrote	`cmake` configure (Ninja generator)
`build/cmake_install.cmake`	Install script	`cmake` configure
`build/compile_commands.json`	Exact compile flags per source file (for IntelliSense) — only when `CMAKE_EXPORT_COMPILE_COMMANDS` is ON	`cmake` configure
`build/generated/app_config.h`	Config header filled in from `config/app_config.h.in`	`configure_file()`
`build/hello.exe`	The compiled app	the build (Ninja)
`build/**/lib*.a`	Static libraries (`libmenus.a` , `libadc.a` , `libmath_utils.a`)	the build
`build/tests/test_*.exe`	Compiled test programs	the build
`build/tests/cmocka.dll`	Copied next to the tests by a POST_BUILD step so they run	the build

With the **Visual Studio** generator instead of Ninja, `build/` would hold `*.sln` and `*.vcxproj` files (and the exe under `build/Debug/`) — same `CMakeLists.txt` , different generated files. That's the generator's job (Chapter 5).

The two "template → generated" pairs you wrote

Some generated files come from a template *you* maintain. You edit the template; CMake writes the output into `build/` :

You edit (committed)	CMake generates (in build/, ignored)	Via
`config/app_config.h.in`	`build/generated/app_config.h`	`configure_file()`
—	`build/compile_commands.json`	`set(CMAKE_EXPORT_COMPILE_COMMANDS ON)`

Never edit the generated `app_config.h` — your changes vanish on the next configure. Change the `.in` template or the CMake `option()` / `set()` values instead.

Regeneration timing

- **Configure-time** artifacts (cache, `build.ninja` , `compile_commands.json` , `app_config.h`) are (re)written when you run `cmake -S . -B build ...` . Changing an `option()` value or a template requires re-running that step — a plain

`cmake --build build` will not regenerate them.

- **Build-time** artifacts (`.exe` , `.a`) are produced by `cmake --build build` and rebuilt incrementally when their sources change.

To start completely fresh

```
rm -rf build
```

Nothing of value is lost — it all regenerates from your committed sources.